PRIVACY PRESERVING WAREHOUSE COORDINATION FOR NATIONAL PADDY DISTRIBUTION

Senarathne S. M. B. V. B.

(IT22372444)

B.Sc. (Hons) Degree in Information Technology, Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

Privacy Preserving Warehouse Coordination for National Paddy Distribution

Senarathne S. M. B. V. B.

(IT22372444)

Dissertation submitted in partial fulfillment of the requirements for the B.Sc. (Hons) Degree in Information Technology, Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	**Student ID**	**Signature**
Senarathne S. M. B. V. B.	IT22372444	

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor: Date:

ABSTRACT

Sri Lanka's Paddy Marketing Board coordinates a dispersed network of regional paddy warehouses through informal channels such as telephone calls and mobile messaging. Stock movements are reported without cryptographic verification, capacity claims made during disaster response cannot be checked, anomalous warehouse behaviour goes undetected, and redistribution decisions leave no independently auditable trail. National food security depends on this network's operational integrity, particularly during seasonal surplus, natural disasters and regional supply imbalance.

This dissertation presents a privacy preserving coordination system that addresses these four gaps through three integrated mechanisms. A zero knowledge proof circuit implemented in Circom using the Groth16 proving scheme allows a warehouse supervisor to prove that reported available capacity meets a required threshold without revealing the capacity value to peer organisations or writing it to the shared ledger. A two layer Graph Attention Network trained on eight behavioural features computes per-warehouse reliability scores by propagating signals across the redistribution graph, so that a warehouse is assessed partly on the behaviour of those it transacts with rather than on its own reported figures alone. A permissioned Hyperledger Fabric blockchain running on a two organisation channel anchors stock events, disaster declarations, redistribution orders and proof verification results as immutable records, producing a timestamped audit trail that persists independently of the application database.

Evaluation indicates that the circuit enforces capacity constraints with zero false acceptances across all invalid inputs; that the Graph Attention Network correctly classified 26 of 30 nodes under in sample evaluation on a synthetic dataset with a 24-node majority class; and that tamper detection is confirmed through SHA-256 hash recomputation, where editing a quantity directly in the database breaks the integrity check while the stored hash remains unchanged. The system has been deployed to a public cloud instance with transport layer security and verified end to end.

The system is presented as a proof-of-concept prototype demonstrating the architectural viability of combining these three mechanisms in a government agricultural logistics context.

Keywords: zero knowledge proofs, graph attention networks, Hyperledger Fabric, paddy warehouse coordination, anomaly detection

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. Mahima Weerasinghe of the Department of Computer Science, Sri Lanka Institute of Information Technology, for her continuous guidance, technical insight and patience throughout this research. Her feedback shaped both the direction of the work and the rigour with which it was evaluated.

I am also deeply thankful to my co-supervisor, Mr. Eishan Weerasinghe of the Department of Software Engineering, for his advice on system design and for the time he gave to reviewing the progress of this component at every stage.

I extend my sincere appreciation to Mr. Suranga Senanayake, our external supervisor, whose knowledge of Paddy Marketing Board operations grounded this work in the realities of warehouse coordination. The requirements described in this dissertation were elicited directly from his experience of the current process.

My heartfelt appreciation goes to the Department of Software Engineering, the Faculty of Computing and Sri Lanka Institute of Information Technology for providing the facilities and academic environment that made this research possible.

I would also like to thank my research group colleagues for their collaboration on system integration and for the many discussions that improved this work.

Finally, I would like to thank my family and friends for their encouragement and support throughout this project.

TABLE OF CONTENTS

1. INTRODUCTION 1

1.1 Background Literature 1

1.1.1 Paddy distribution and warehouse coordination in Sri Lanka 1

1.1.2 Blockchain based agricultural traceability and coordination 2

1.1.3 Zero knowledge proof applications in supply chain privacy 3

1.1.4 Graph neural networks for anomaly detection 4

1.1.5 Integrated smart contract and artificial intelligence coordination frameworks 5

1.2 Research Gap 6

1.3 Research Problem 7

1.4 Research Objectives 8

1.4.1 Main objective 8

1.4.2 Specific objectives 9

2. METHODOLOGY 11

2.1 Methodology 11

2.1.1 System architecture overview 11

2.1.2 Permissioned blockchain layer and chaincode design 13

2.1.3 Zero knowledge proof pipeline 16

2.1.4 Graph attention network for reliability scoring 18

2.1.5 Disaster response coordination and redistribution ranking 21

2.1.6 Mobile application implementation 23

2.1.7 Backend API and deployment infrastructure 25

2.2 Commercialization Aspects of the Product 27

2.2.1 Market opportunity 27

2.2.2 Revenue model 28
2.2.3 Regulatory considerations 29
2.2.4 Data privacy and security 30
2.3 Testing and Implementation 30
2.3.1 Unit testing 31
2.3.2 Integration and system testing 32
2.3.3 Non-functional requirement measurements 33
2.3.4 Known limitations and mitigations 34
3. RESULTS AND DISCUSSION 35
3.1 Results 35
3.1.1 Zero knowledge proof circuit evaluation 35
3.1.2 Graph attention network evaluation 35
3.1.3 Blockchain audit trail and integrity verification 36
3.1.4 Deployment results 37
3.2 Research Findings 38
3.3 Discussion 42
3.4 Summary of the Student's Contribution 44
4. CONCLUSION 46
4.1 Summary 46
4.2 Future Work 47
4.3 Conclusion 48
5. REFERENCES 50
6. GLOSSARY 52
7. APPENDICES 54
Appendix A: Zero Knowledge Circuit Source 54
Appendix B: Graph Attention Network Model Definition 54

Appendix C: Chaincode Asset Definitions 56

Appendix D: Test Execution Output 57

Appendix E: Deployment Verification Evidence 59

LIST OF TABLES

Table 1: Capability comparison of related approaches

Table 2: Warehouse node feature vector definition

Table 3: Composite redistribution ranking weights

Table 4: Revenue model tier comparison

Table 5: Unit test coverage by module

Table 6: System test cases and outcomes

Table 7: Non-functional requirement measurements

Table 8: Known limitations and mitigations

Table 9: Graph Attention Network training progression

Table 10: Integrity verification results under tampering

Table 11: Summary of the student's contribution

LIST OF FIGURES

Figure 1: System architecture of the warehouse coordination component

Figure 2: Hyperledger Fabric transaction endorsement and commit flow

Figure 3: Zero knowledge proof generation and verification pipeline

Figure 4: Synthetic warehouse graph used for Graph Attention Network training

Figure 5: Disaster response and redistribution workflow

Figure 6: Regional Manager dashboard

Figure 7: Blockchain audit trail screen

Figure 8: Disaster redistribution ranking screen

Figure 9: Deployed container topology

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
CA	Certificate Authority
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
ELU	Exponential Linear Unit
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GDPR	General Data Protection Regulation
GNN	Graph Neural Network
gRPC	Google Remote Procedure Call
JSON	JavaScript Object Notation
JWT	JSON Web Token
MSP	Membership Service Provider
NFR	Non-Functional Requirement
ORM	Object Relational Mapping
PMB	Paddy Marketing Board
R1CS	Rank-1 Constraint System
REST	Representational State Transfer
RM	Regional Manager
SHA-256	Secure Hash Algorithm, 256-bit
TLS	Transport Layer Security
UAT	User Acceptance Testing
WASM	WebAssembly
ZKP	Zero Knowledge Proof

1. INTRODUCTION

Sri Lanka's Paddy Marketing Board (PMB) is the central government body responsible for purchasing, storing and distributing rice through a dispersed network of regional warehouses. Rice is the country's staple food, and national food security depends directly on the operational integrity of this network, particularly during seasonal surplus, natural disasters and regional supply imbalance. The Board's mandate and network structure are documented in its annual reporting [1].

Despite this significance, PMB warehouse coordination still relies on unofficial channels such as telephone calls and mobile messaging for reporting stock movements, announcing disasters and organising inter-warehouse redistribution. This was reported by a serving PMB Regional Manager consulted during requirements gathering for this study. The dependency introduces four compounding vulnerabilities that the present work addresses.

Supervisor stock event reports lack cryptographic verification, making fraudulent quantity reporting undetectable by later audit. A warehouse cannot demonstrate that its available capacity meets a threshold without revealing its precise stock level, so Regional Managers must select redistribution sources from self-reported figures alone. The network lacks any systematic method for identifying unusual behavioural trends such as persistently high damage claim rates or implausible inflow-to-utilisation disparities. And redistribution decisions are not recorded in any immutable, independently verifiable form, leaving post-hoc auditing dependent on informal records susceptible to alteration or loss.

1.1 Background Literature

1.1.1 Paddy distribution and warehouse coordination in Sri Lanka

The Paddy Marketing Board operates as a buyer of last resort in the Sri Lankan paddy market, purchasing from farmers at a guaranteed price during harvest seasons and releasing stock to stabilise consumer prices during shortage. Its warehouse network

spans the principal cultivation districts, with individual facilities differing substantially in storage capacity, construction quality and susceptibility to flooding.

The coordination problem this creates is distinct from the traceability problem that dominates agricultural blockchain literature. Traceability asks where a consignment came from; coordination asks which facility should hold stock now, and which facility should receive it when another is compromised. The second question requires decisions to be made quickly, on information supplied by the parties whose interests those decisions affect, and recorded in a form that survives dispute.

Kamble et al. [6] observe that research in agricultural supply chains concentrates on traceability, provenance and production analytics, while downstream logistics coordination receives considerably less coverage. Yogarajan et al. [14] confirm this gap in a systematic review of blockchain adoption in agriculture, finding that the dominant application patterns address upstream verification rather than inter-facility stock movement. Negi [19] reaches a similar conclusion for humanitarian logistics, where blockchain work has likewise concentrated on donation and fund traceability rather than inter-facility redistribution.

1.1.2 Blockchain based agricultural traceability and coordination

Blockchain has been widely applied to agricultural supply chain transparency for tamper evident record keeping and provenance verification. Tian [2] combined radio-frequency identification with a permissioned blockchain for food safety traceability, demonstrating how immutable records remove the single point of failure inherent in centralised agricultural databases. Mondal et al. [3] extended this by binding physical items to ledger records through tag readings, establishing a pattern of coupling a custody event to an immutable digital record that the present work applies to warehouse stock events.

The architectural distinction that matters for this research is between recording and deciding. Traceability systems record what has already happened to a consignment. A coordination system must additionally record the basis on which a decision was taken, so that the decision itself can be audited. Cryptographic audit trail mechanisms for

redistribution decisions are, to the best of the author's knowledge, absent from this literature.

Critical evaluation: the recording-to-deciding gap

Existing agricultural blockchain deployments treat the ledger as a passive record of transactions that occurred elsewhere. None of the reviewed systems anchors the inputs to an operational decision, the decision itself, and the verification of claims made during that decision as linked on chain assets. This is the gap the present work closes by anchoring disaster declarations, redistribution orders and proof verification results as chaincode managed assets, permitting audit queries independent of the application database.

1.1.3 Zero knowledge proof applications in supply chain privacy

Ben-Sasson et al. [7] established the practicality of succinct non-interactive arguments of knowledge, laying the groundwork for Groth16 [8], the proving scheme used in this work, which produces constant size proofs of three group elements verifiable through a single pairing product equation. The property that makes this family of schemes relevant to warehouse coordination is that a statement about a private value can be verified without the value being disclosed.

Narula et al. [4] applied zero knowledge balance proofs to distributed-ledger auditing, confirming aggregate properties of private holdings without revealing them. Choudhuri et al. [9] establish that a blockchain can serve as a public setup for secure computation. The present work applies these principles through a comparator circuit proving that a private capacity value is greater than or equal to a public threshold, producing a three signal proof verifiable on chain through Fabric chaincode.

Critical evaluation: proofs without an operational context

Prior applications of zero knowledge proofs in supply chain settings demonstrate the cryptographic mechanism but do not situate it within a decision process that depends on the result. A capacity proof is only useful if a Regional Manager can act on it, which requires the verification outcome to be recorded, attributed and ranked alongside other

candidates. The present work treats proof verification as an input to a ranking function rather than as an end in itself.

1.1.4 Graph neural networks for anomaly detection

Graph Neural Networks perform well in anomaly detection where relational structure supplies discriminative information unavailable from node level characteristics alone. Kipf and Welling [10] presented the Graph Convolutional Network formulation, in which a node representation is computed as a normalised aggregate of its neighbours. Veličković et al. [11] extended this with attention mechanisms that learn adaptive edge weights during message passing.

The distinction is material to this problem. In a Graph Convolutional Network every neighbour contributes equally; in a Graph Attention Network the model learns which neighbours matter. The significance of a redistribution link between two warehouses is not uniform across the network, since a single emergency transfer carries different evidential weight from a sustained bilateral relationship.

Dou et al. [5] showed that GNN based fraud detection in financial transaction graphs significantly outperforms feature based classifiers by capturing collusive patterns across connected accounts, a situation directly analogous to the collusive redistribution pattern addressed in this work. The attention based design is further motivated by Liu et al. [12], who show that fraud detection GNNs degrade when neighbour relevance is inconsistent across a graph, supporting adaptive rather than uniform neighbour weighting.

Critical evaluation: relational signals in logistics networks

Graph structured anomaly detection has been applied extensively to financial transaction networks and comparatively little to physical logistics networks, where the edges represent movements of goods rather than movements of money. The behavioural signatures differ: a warehouse cannot transfer stock it does not hold, so an implausible redistribution pattern is constrained by physical capacity in a way a financial transfer is not. Whether this constraint makes the detection problem easier or harder is not established in the reviewed literature.

1.1.5 Integrated smart contract and artificial intelligence coordination frameworks

Salah et al. [13] identify a separation between off chain inference and on chain result anchoring as the appropriate pattern for production deployment, a conclusion driven by the computational cost and non-determinism of running model inference inside a smart contract. The same separation is followed in this work: inference runs in an isolated Python service, while reliability scores are persisted relationally and referenced during ranking rather than anchored individually.

Motie and Raahemi [15] survey financial fraud detection using graph neural networks and note that integration with transactional infrastructure remains largely theoretical. Ernstberger et al. [16] provide the zk-Bench toolset for comparative evaluation of succinct argument implementations, establishing that performance characterisation of proving systems requires instrumented benchmarking rather than wall clock timing on a single machine. Melo et al. [17] and Shuaib et al. [18] provide established methodologies for Hyperledger Fabric performance evaluation, modelling transaction flow and endorsement policies.

A comparison of the reviewed approaches against the capabilities required for the present problem is given in Table 1. To the best of the author's knowledge, no previous system combines permissioned blockchain audit trails, graph structured anomaly detection and zero knowledge capacity verification within a single agricultural warehouse coordination architecture.

Work	**Immutable audit trail**	**ZK capacity proof**	**Graph anomaly detection**	**Warehouse coordination**	**Disaster response**
Tian [2]	Yes	No	No	No	No
Narula et al. [4]	Yes	Yes	No	No	No
Dou et al. [5]	No	No	Yes	No	No
Salah et al. [13]	Yes	No	Yes	No	No
Yogarajan et al. [14]	Yes	No	No	Partial	No
Negi [19]	Yes	No	No	No	No
This work	Yes	Yes	Yes	Yes	Yes

Table 1: Capability comparison of related approaches

Capabilities in Table 1 are assessed against the requirements of the present problem, not against the stated objectives of the cited works. A cell marked "No" indicates that the capability is outside that work's scope rather than that the work attempted and failed to provide it.

1.2 Research Gap

The literature reviewed above leaves six gaps that this research addresses.

Firstly, agricultural blockchain research concentrates on upstream traceability and provenance verification, while downstream inter-facility coordination receives comparatively little attention [6], [14]. The decision of which warehouse should receive stock during an emergency, and the record of why that warehouse was chosen, falls outside the scope of existing traceability architectures.

Secondly, capacity disclosure during coordination is treated as unavoidable. Where a warehouse must demonstrate that it can absorb a transfer, existing systems require it to report its stock level, exposing commercially and politically sensitive operational data to peer organisations. Zero knowledge techniques capable of resolving this have been demonstrated for ledger auditing [4] but not applied to operational capacity claims in logistics.

Thirdly, anomaly detection in agricultural supply chains, where it exists, operates on individual records rather than on relational structure. A warehouse reporting an implausible damage ratio is detectable by threshold; a group of warehouses coordinating transfers inconsistent with legitimate emergency response is not. Graph structured detection has been shown effective for this class of pattern in financial networks [5], [12] but has not been applied to warehouse redistribution graphs.

Fourthly, where blockchain and machine learning are combined in supply chain research, the integration is typically conceptual. Salah et al. [13] identify the correct architectural separation, but implementations that anchor model outputs, cryptographic verification results and operational decisions within one channel are rare.

Fifthly, redistribution decision trails are not independently auditable. Even where stock movements are recorded on a ledger, the parameters of the decision (which candidates were considered, what each claimed, which claims were cryptographically verified) are held only in the application database and can be altered without detection.

Finally, the reviewed systems are evaluated in isolation rather than as deployed software. Proving systems are benchmarked, models are scored on held out data, and blockchain networks are load tested, but the composition of all three into a running system with authenticated role based access and a client application is seldom demonstrated.

These limitations highlight the need for a warehouse coordination architecture that records decisions as well as events, verifies capacity claims without disclosing them, detects anomalous behaviour from relational signals, and is demonstrated as an integrated deployed system rather than as three separate mechanisms.

1.3 Research Problem

Paddy warehouse coordination in Sri Lanka is conducted through channels that produce no verifiable record. When a Regional Manager telephones a warehouse supervisor to ask whether stock can be accepted, the answer is unverifiable, the decision that follows is unrecorded, and the resulting transfer is reconciled afterwards from documents held by the parties themselves. This is tolerable in ordinary operation and fails precisely when it matters most, during a flood or a storage failure, when decisions are made quickly and scrutinised later.

Several barriers prevent the obvious remedies from working. First, a centralised database controlled by one organisation does not resolve a trust problem between organisations, because the controlling party can alter it. Second, requiring warehouses to publish stock levels to a shared system creates a disclosure problem: capacity figures carry political and commercial sensitivity, and a system that mandates publication will be resisted or populated with inaccurate figures. Third, threshold based monitoring cannot detect misreporting that is individually plausible but collectively anomalous, because each report falls within acceptable bounds in isolation.

The coordination problem is additionally shaped by heterogeneity. Warehouses differ in capacity by an order of magnitude, in flood exposure by district, and in reporting discipline by supervisor. A rule calibrated to one facility misclassifies another, which is why a learned, relational assessment is preferable to a fixed threshold.

A practical solution would involve a warehouse coordination system that:

1. records stock events, disaster declarations and redistribution orders on an immutable, multi-organisation ledger whose contents can be audited without reliance on the application database;
2. allows a warehouse to prove that its available capacity meets an operational threshold without disclosing the capacity value to peer organisations or writing it to the shared ledger;
3. identifies anomalous warehouse behaviour from relational signals propagated across the redistribution network rather than from individual metric thresholds; and
4. presents these capabilities through role separated interfaces usable by administrators, regional managers, warehouse supervisors and auditors.

A system meeting these requirements would give the Paddy Marketing Board a coordination process whose decisions are defensible after the fact, whose participants are not required to disclose sensitive operational figures, and whose anomalies surface before rather than after an audit. The wider significance is that the same architecture applies to any government logistics network in which facilities belonging to different administrative units must coordinate under conditions of partial trust.

1.4 Research Objectives

1.4.1 Main objective

This component aims to design, implement and evaluate a privacy preserving warehouse coordination system for Sri Lanka's national paddy distribution network, integrating a permissioned Hyperledger Fabric blockchain for tamper evident audit trails, a Groth16 zero knowledge proof circuit for capacity verification without

disclosure, and a Graph Attention Network for relational anomaly detection, exposed through a role separated mobile application. The scope of this dissertation is the warehouse coordination component in its entirety: the ledger layer, the proof system, the anomaly detector, the backend services, the mobile client and the deployment.

1.4.2 Specific objectives

The warehouse coordination component achieves its main objective through the following technical implementations:

1. To conduct a literature review of blockchain based agricultural traceability, zero knowledge proof applications in supply chain privacy, graph neural network anomaly detection and integrated smart contract coordination frameworks, in order to establish the capability gap addressed by this work.
2. To design and deploy a permissioned Hyperledger Fabric network on a two organisation channel representing PMB headquarters and the Ministry of Agriculture, using CouchDB as the world state database to support the rich JSON queries required for warehouse history and audit trail retrieval.
3. To implement chaincode in Go defining asset types for stock events, disaster declarations, redistribution orders and proof verification results, each type prefixed to enable selector queries without full ledger scans.
4. To implement a zero knowledge proof circuit in Circom 2.2.3 using the Groth16 proving scheme and the GreaterEqThan comparator, proving that a private available capacity value meets a public threshold, and to integrate proof generation and verification into the disaster response workflow.
5. To design an eight dimensional behavioural feature vector for warehouse nodes, derived from stock event and redistribution records, capturing damage ratios, inflow patterns, utilisation rates and redistribution frequencies.
6. To implement and train a two layer Graph Attention Network with four attention heads in the first layer, computing per-warehouse reliability scores from features propagated across the redistribution graph, and to expose inference through a dedicated model service.

7. To implement a composite ranking function combining geographic distance, available capacity and GNN derived reliability, so that a Regional Manager responding to a disaster is presented with candidate warehouses ordered by operational suitability.
8. To implement SHA-256 document integrity verification in which the digest is recomputed from the stored record fields at verification time rather than compared against a second stored value, so that direct modification of a quantity is detectable.
9. To develop a React Native mobile client presenting role separated interfaces for administrators, regional managers, warehouse supervisors and auditors, with authentication through short lived access tokens and rotating refresh tokens.
10. To deploy the complete system to a public cloud instance with transport layer security, and to evaluate it through unit testing, system testing, non-functional requirement measurement and end-to-end verification of ledger anchoring and integrity checking.

2. METHODOLOGY

2.1 Methodology

2.1.1 System architecture overview

The system follows a microservice aligned layered architecture in which each capability runs as an independent service orchestrated by a central Node.js backend, implemented in TypeScript with Express and connected to a MySQL database through an object relational mapping layer. All client facing API endpoints are authenticated using JSON Web Token pairs, comprising a short lived access token and a long lived refresh token with rotation and replay detection, and authorised through role based middleware enforcing four roles: Administrator, Regional Manager, Warehouse Supervisor and Auditor.

Presentation tier

A React Native mobile client built with Expo presents a distinct interface per role. Supervisors record stock events and upload supporting documents; Regional Managers declare disasters, review ranked redistribution candidates and issue orders; Administrators register warehouses and users; Auditors hold read-only access to the blockchain audit trail and integrity verification. Role determination is derived from the authenticated token rather than from client side state, so an interface cannot be reached by manipulating navigation.

Application tier

The Node.js backend owns all business logic and is the only component permitted to write to the database or to submit transactions to the ledger. A separate Python service hosts the Graph Attention Network and exposes inference over HTTP, following the off chain inference pattern identified by Salah et al. [13]. Keeping model inference in its own process isolates the Python runtime and its machine learning dependencies from the request handling path of the main API.

Data tier

MySQL holds the operational relational state: warehouses, users, stock events, disasters, redistribution orders and documents. Hyperledger Fabric holds the immutable record, with CouchDB as the world state database. The two are deliberately not mirrors of one another. The relational database answers operational questions quickly; the ledger answers the question of whether the relational record has been altered.

The component follows this end-to-end pipeline when a supervisor submits a stock event:

1. the request is authenticated and the caller's role verified against the target warehouse;
2. a database transaction opens and an exclusive row lock is taken on the warehouse record;
3. current stock is computed by aggregating prior inflow and outflow events, and the proposed event is validated against capacity;
4. a SHA-256 digest is computed over a canonical serialisation of the warehouse identifier, event type, quantity, reporting user and timestamp;
5. the event is written with that digest and the same timestamp value used in its computation, and the transaction commits;
6. the event is submitted to Fabric chaincode as a stock event asset and the returned reference persisted; and
7. the updated warehouse summary is returned to the client.

The ordering of steps five and six is significant. The ledger submission occurs after the database transaction commits, so that a slow or unavailable peer cannot hold a row lock open. The consequence, examined in Section 3.2, is that the two stores can diverge if anchoring fails.

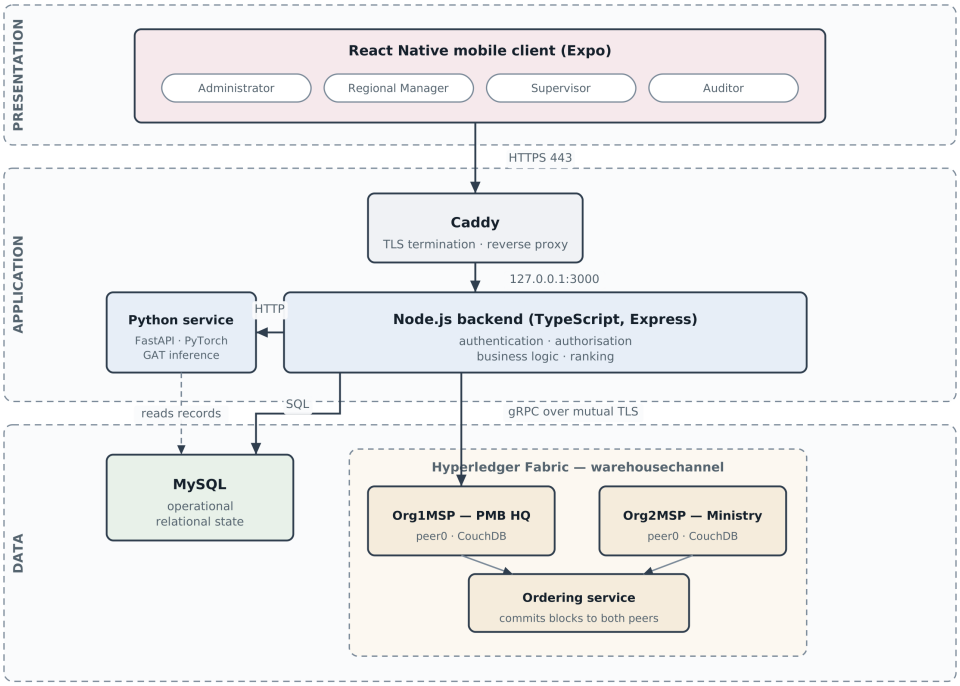


Figure 1: System architecture of the warehouse coordination component

2.1.2 Permissioned blockchain layer and chaincode design

Hyperledger Fabric was chosen over public alternatives for three reasons. First, deterministic transaction finality: an ordering service commits blocks without probabilistic confirmation, which matters when a redistribution order must be acted upon immediately. Second, a permissioned identity model aligning with the government organisational hierarchy, where participants are known institutions rather than anonymous addresses. Third, endorsement policies enabling multi-organisation consensus without proof-of-work overhead, which is unjustifiable for a government logistics network.

The network is deployed as a two organisation channel named warehousechannel. Org1MSP represents Paddy Marketing Board headquarters and Org2MSP the Ministry of Agriculture. Each organisation runs one peer with its own CouchDB world state database and its own certificate authority. CouchDB was selected over LevelDB

because warehouse history and audit trail retrieval require rich JSON queries over asset fields, which LevelDB's key-value interface cannot express without full ledger scans.

Chaincode asset model

The chaincode, written in Go, defines four asset types, each stored under a type prefixed key enabling selector queries:

- **Stock event assets** record the warehouse identifier, event type, quantity in tons, document hash, reporting user, the submitting organisation's membership service provider identity and a timestamp derived from the transaction context.
- **Disaster declaration assets** record the affected warehouse, disaster type, severity, declaring user and declaration time.
- **Redistribution order assets** record source and destination warehouses, quantity, the associated disaster, and an integrity digest computed as the SHA-256 hash of the concatenated order fields.
- **Proof verification result assets** record the outcome of a zero knowledge capacity proof, storing SHA-256 hashes of the proof and its public signals rather than the full proof data.

The redistribution order integrity digest allows an auditor to verify that order parameters were not altered off chain after anchoring. It is not a digital signature and provides no non-repudiation: any party reconstructing the same field values produces the same digest. Attribution is therefore layered. The submitting organisation is identified on chain through the membership service provider identity derived from the transaction context, while the individual user is identified only relationally. Per-user non-repudiation would require enrolment certificates issued by each organisation's certificate authority, identified as future work.

Gateway integration and timestamp determinism

The backend connects to the Fabric peer through the gateway software development kit using a singleton gateway pattern, authenticating with administrator certificates from the Org1 membership service provider and performing transport layer security verification against the peer's certificate authority. Submit transactions invoke

chaincode write functions and await orderer commitment; evaluate transactions invoke read functions without creating on chain records.

An endorsement consistency issue arose during implementation. Chaincode initially generated timestamps from the host system clock, which differs between endorsing peers; because each peer executes the transaction independently and the read-write sets must match, the resulting proposals diverged and endorsement failed. The resolution was to derive all timestamps from the ledger supplied transaction context, which is identical across endorsers by construction. This is a general constraint on chaincode design: any source of non-determinism, including clocks, random values and external calls, breaks endorsement.

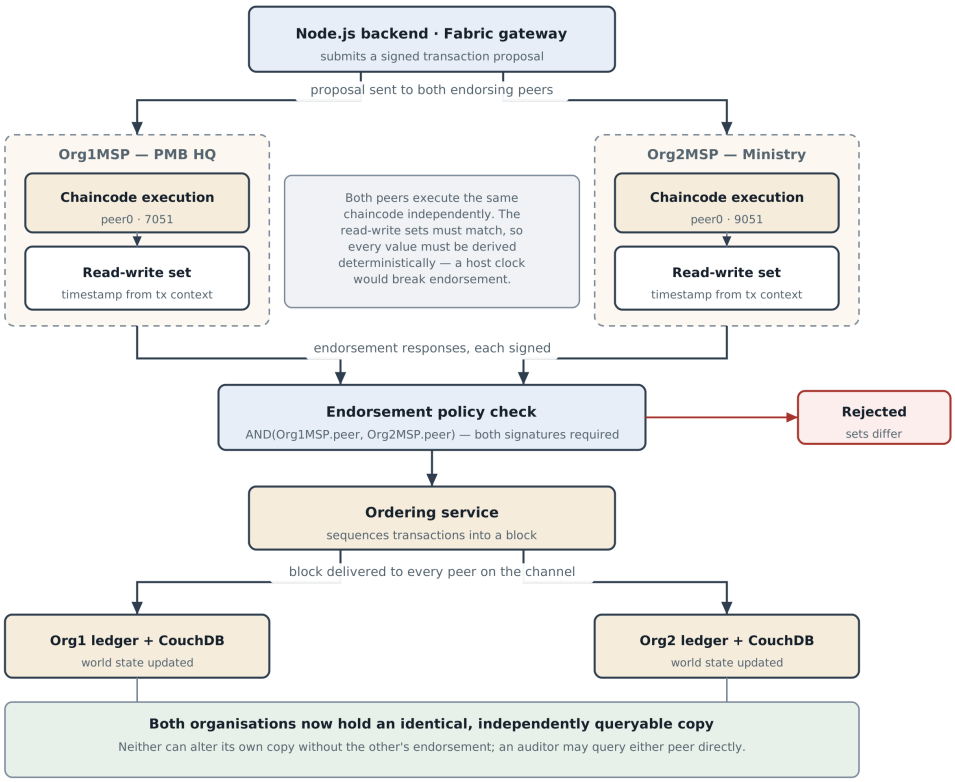


Figure 2: Hyperledger Fabric transaction endorsement and commit flow

2.1.3 Zero knowledge proof pipeline

The second gap is addressed by a zero knowledge proof component allowing a warehouse to demonstrate that available capacity meets a disaster response threshold without revealing precise stock levels. The circuit is written in Circom 2.2.3 using the GreaterEqThan comparator from circomlib, which constrains the comparison to values representable in 32 bits and therefore supports quantities far exceeding any realistic warehouse capacity.

Circuit construction

The circuit defines one private input, availableCapacity, and two public inputs, threshold and warehouseCommitment. The output signal valid is constrained to equal one, so proof generation fails at the circuit level if availableCapacity is less than threshold. This is the critical security property: a dishonest warehouse cannot generate a valid proof for capacity it does not possess, because the constraint is enforced by the Rank-1 Constraint System before the proving key is applied. Failure occurs during witness generation, not at verification, so an invalid claim never produces a proof to submit.

The three public signals are valid, threshold and warehouseCommitment; availableCapacity appears in no public output. The warehouseCommitment signal is declared as a public input but is not constrained within the circuit in the current implementation. Warehouse identity is instead bound at the application layer, with the submission endpoint associating each verified proof with the warehouse and disaster identifiers of the authenticated request and rejecting duplicates. This is a limitation: the proof does not attest to which warehouse produced it, leaving that binding dependent on the application server rather than on the proof system. A circuit level constraint of the form Poseidon(warehouseId, availableCapacity, salt) equals warehouseCommitment, with warehouseId and salt as additional private inputs, would relocate the guarantee into the proof. Poseidon is preferred over an in circuit SHA-256, which costs tens of thousands of constraints against a few hundred. This is identified as immediate future work.

Trusted setup and proof lifecycle

The trusted setup follows the Groth16 two phase ceremony: a Powers of Tau at depth fourteen, followed by a circuit specific phase two contribution producing the proving key, with the verification key loaded once at startup. Proof generation is currently server side. On successful verification the result is persisted relationally and anchored on the ledger, which stores SHA-256 hashes of the proof and its public signals rather than the full proof data.

Threat model and privacy scope

The guarantee does not hold uniformly against all parties. The application server learns availableCapacity, since proof generation executes server side, and the Regional Manager observes capacity through the threshold term in the ranking function; neither is treated as an adversary here. The value is never written to the ledger and appears in no public signal, so it is not disclosed to the second endorsing organisation, to peer warehouses, or to any auditor querying the ledger. The property claimed is therefore confidentiality with respect to the shared ledger and to other organisations, not with respect to the application server. Migrating proof generation to browser side WebAssembly using the compiled artifact would extend the guarantee to the server, and is identified as future work.

A second limitation concerns threshold probing. Because the threshold is public, an adversary able to request proofs at chosen thresholds could recover the capacity by binary search in logarithmic time. Two mitigations apply: one accepted proof per warehouse and disaster pair, with duplicates rejected; and rate limiting on all authenticated endpoints. Restricting thresholds to fixed capacity bands would bound disclosure to the band boundaries regardless of query volume, and is not implemented here.

Finally, the circuit attests that the supplied value satisfies the threshold, not that it corresponds to physical inventory. Establishing that would require the private input to derive from an authenticated record, for example a Merkle inclusion proof against a periodically anchored stock ledger root verified in circuit. The prototype therefore prevents disclosure of the reported capacity but does not prevent a supervisor from

reporting an inflated figure. Detection of that class of misreporting is delegated to the anomaly component described next, which operates over reporting patterns rather than individual claims. The two mechanisms are consequently complementary: the proof addresses disclosure, the anomaly detector addresses truthfulness.

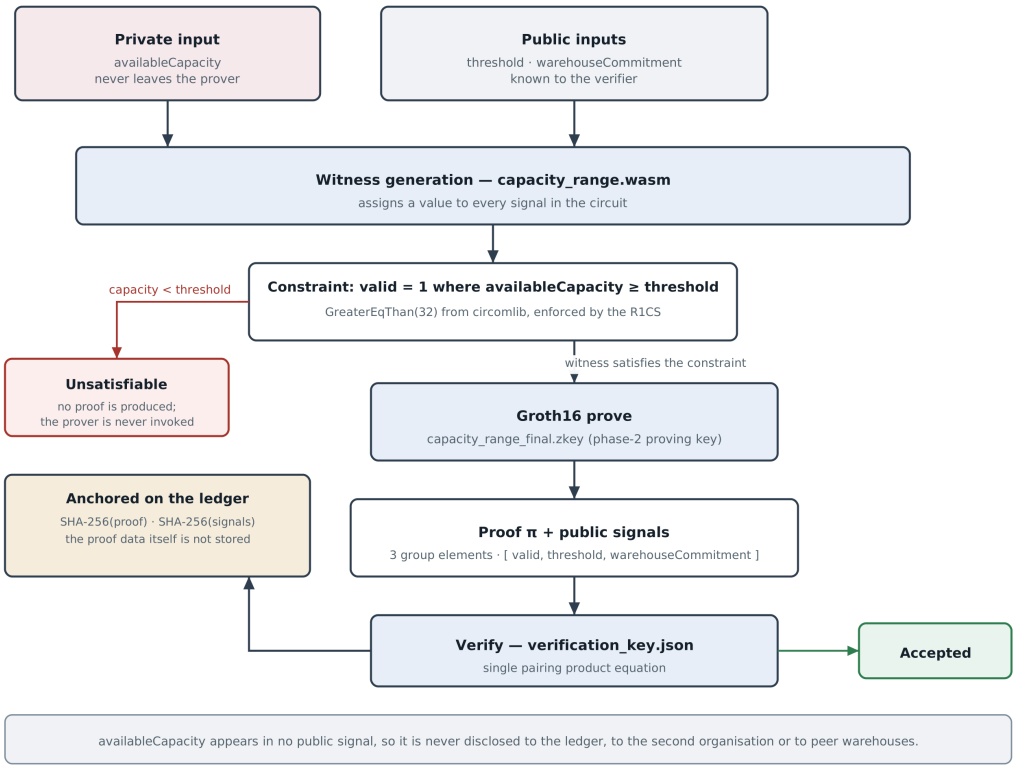


Figure 3: Zero knowledge proof generation and verification pipeline

2.1.4 Graph attention network for reliability scoring

The third gap concerns detection of anomalous behavioural patterns that individual metric thresholds cannot identify in isolation. The system constructs a homogeneous graph in which nodes represent active warehouses and directed edges represent historical redistribution orders, with node features computed from stock event and redistribution records into the eight dimensional vector defined in Table 2.

Index	Feature name	Formula
f0	total_events	min(count / 100, 1.0)
f1	inflow_ratio	inflow_events / total_events

Index	Feature name	Formula
f2	damage_ratio	damage_events / total_events
f3	utilization_pct	current_stock / capacity
f4	avg_quantity	min(avg_tons_per_txn / 500, 1.0)
f5	event_frequency	min((total_events / days_active) / 10, 1.0)
f6	has_disaster	1.0 if active disaster else 0.0
f7	redistribution_count	min(outgoing_orders / 10, 1.0)

Table 2: Warehouse node feature vector definition

Current stock is computed as the sum of inflow tons minus the sum of outflow tons, using the same formula as the backend stock service. Days active is defined as the maximum of one and the number of days since warehouse creation, preventing division by zero. The normalisation denominators are order-of-magnitude scaling constants mapping each feature into approximately the unit interval; they were not tuned against operational data and would require recalibration on real data.

Anomaly patterns and synthetic data

Three anomaly patterns are detectable in combination, corresponding to the classes injected into the synthetic dataset. Excessive damage claiming appears as f2 in the range 0.3 to 0.7. Capacity overclaiming appears as high f1 with near zero f3, indicating stock reported as received and simultaneously as lost without ever being held. Collusive redistribution appears as f7 between 0.7 and 1.0 with elevated f5, suggesting coordinated transfers inconsistent with legitimate emergency response. The remaining features are contextual: f0 identifies dormant warehouses, f4 flags implausible volumes, and f6 prevents false penalisation of warehouses legitimately active in disaster response.

These ranges are configuration parameters of the data generator chosen to produce a separable synthetic signal, reported for reproducibility rather than as empirically grounded thresholds. Empirical loss distributions were unavailable for the PMB network, which is the principal constraint on this component’s evaluation and is discussed in Section 5.

Model architecture

The model comprises two graph attention convolution layers. The first maps the eight dimensional input to sixteen dimensions across four attention heads, giving a sixty-four-dimensional concatenated output; the second reduces this to an eight dimensional embedding with a single head. A two layer perceptron with rectified linear activation and dropout at probability 0.3 maps each embedding to a reliability score in the unit interval through a sigmoid unit.

The attention coefficient for edge (i, j) is computed as the softmax over the neighbourhood of a LeakyReLU activated inner product between a learnable attention vector and the concatenated linear transformations of the two node feature vectors [11]. Each node’s updated representation is then the exponential linear unit activation of the weighted sum of its transformed neighbours, where the neighbourhood includes the node itself through self-loops added at each attention layer.

The justification for graph structured modelling over per-warehouse classification is that a node’s representation is influenced by its neighbours’ behavioural profiles. Consider an established warehouse with a redistribution edge to a newly opened warehouse exhibiting the capacity overclaiming pattern. After applying the attention mechanism the established warehouse attends to the new one with a learned weight, blending part of the latter’s transformed representation into its own embedding, and its reliability score falls relative to its score in isolation. A warehouse’s redistribution connections therefore influence its score in addition to its own reported figures. This is an illustrative trace of model behaviour on two nodes, not an experimental comparison against non-relational classifiers; no such comparison was conducted.

Training configuration

The model is trained on a synthetic dataset of thirty warehouse nodes, twenty-four labelled reliable and six anomalous, generated with fixed random seeds for reproducibility. Training runs for 200 epochs using the Adam optimiser at learning rate 0.01 with weight decay 5×10^{-4}, minimising binary cross entropy loss over all nodes simultaneously. At inference the service reads live warehouse and redistribution records, constructs the current graph and passes it to the trained model.

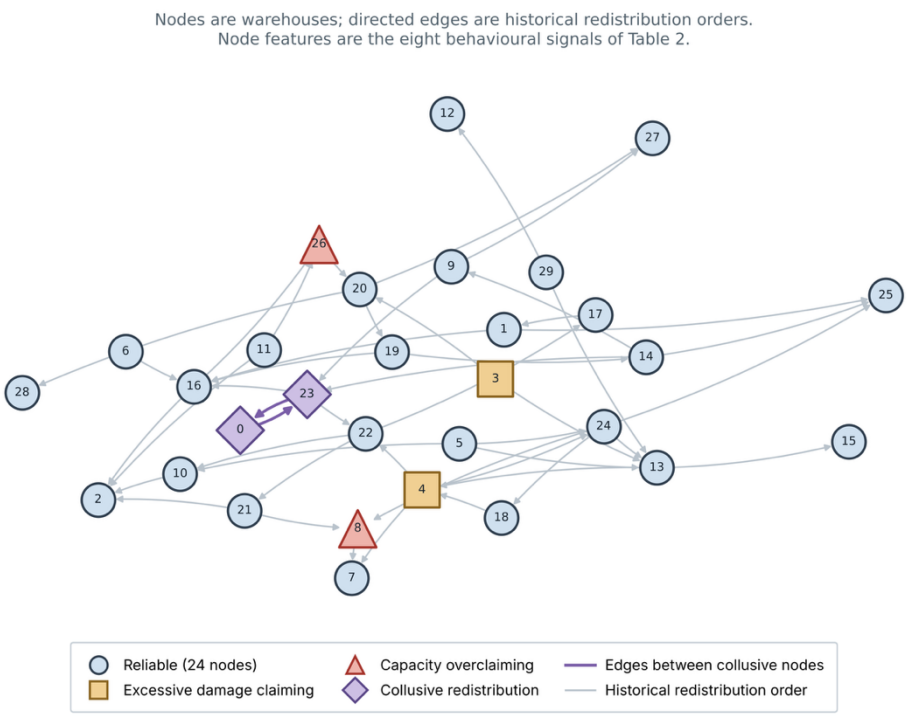


Figure 4: Synthetic warehouse graph used for Graph Attention Network training

2.1.5 Disaster response coordination and redistribution ranking

When a Regional Manager declares a disaster at a warehouse, the system ranks candidate destination warehouses using a composite score integrating the three mechanisms described above:

$$S = 0.40 \cdot d + 0.35 \cdot c + 0.25 \cdot r$$

where d is the normalised inverse distance score computed from the Haversine formula over warehouse coordinates, c is the normalised available capacity score, r is the GNN derived reliability score, and S is the composite selection score.

Term	Weight	Rationale
Distance (d)	0.40	Delivery time is the binding constraint during emergency response
Capacity (c)	0.35	A proximate warehouse without room cannot fulfil an order
Reliability (r)	0.25	A modifier discriminating between logistically comparable candidates

Table 3: Composite redistribution ranking weights

The weights were assigned by the author to reflect operational priority in disaster logistics rather than derived empirically. No sensitivity analysis was conducted, and the ordering they produce has not been validated against decisions a Regional Manager would make unaided. This is a limitation of the current design.

Evacuation direction

A design correction made during implementation is worth recording. The initial model treated redistribution as moving stock towards the affected warehouse, with candidates ranked by free capacity. This inverted the operational intent. When a warehouse is flooded, the objective is to move stock out of it before further loss, not into it. The implementation was revised so that a redistribution order creates an outflow leg at the affected warehouse and an inflow leg at the destination, within a single database transaction that locks both warehouse rows in sorted identifier order to avoid deadlock. The ranking metric was correspondingly changed from free capacity to the quantity each candidate can absorb.

The full disaster lifecycle (declaration, proof verification, redistribution ordering and resolution) is recorded in a chronological audit timeline exposed through a dedicated endpoint, assembling relational records combined with on chain assets retrieved via chaincode evaluate transactions. This gives auditors a single view cross referencing off chain data against independently verifiable blockchain records.

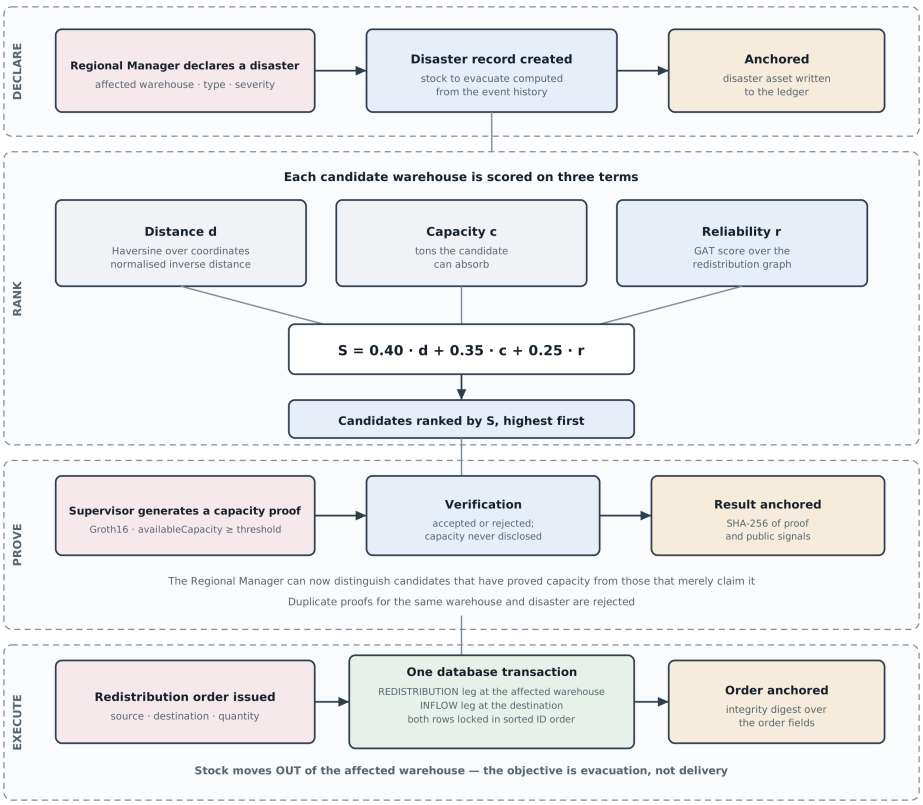


Figure 5: Disaster response and redistribution workflow

2.1.6 Mobile application implementation

Technology stack

The client is a React Native application built with Expo, using file based routing through Expo Router and route groups to separate the four role interfaces. Styling uses React Native StyleSheet with a centralised theme module rather than a utility class framework, a decision taken after dependency conflicts between the utility framework and the Expo software development kit version proved unresolvable without downgrading the toolchain. Interface strings are resolved through a language context supporting Sinhala, Tamil and English.

Authentication and token handling

The client holds a short lived access token and a long lived refresh token. On receiving an unauthorised response the API layer attempts a single token refresh and replays the original request, queuing concurrent requests during the refresh so that a burst of

parallel calls produces one refresh rather than many. Tokens are read from secure storage rather than from component state, so a token refreshed in one screen is immediately effective in all others.

Role separated interfaces

Warehouse Supervisors are routed directly to the detail screen of their assigned warehouse, since the assignment is held on the user record and a supervisor has no legitimate view of other facilities. Regional Managers access the warehouse list, the disaster declaration flow, the ranked candidate screen and the redistribution order form. Administrators register warehouses, including coordinate selection through an address search that resolves to latitude and longitude, and manage user accounts. Auditors receive a read-only interface over the blockchain audit trail and the integrity verification endpoint.

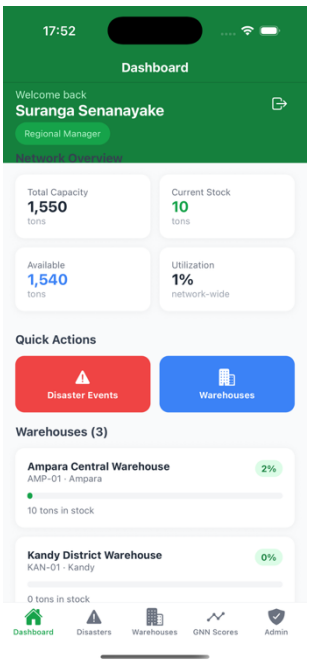


Figure 6: Regional Manager dashboard

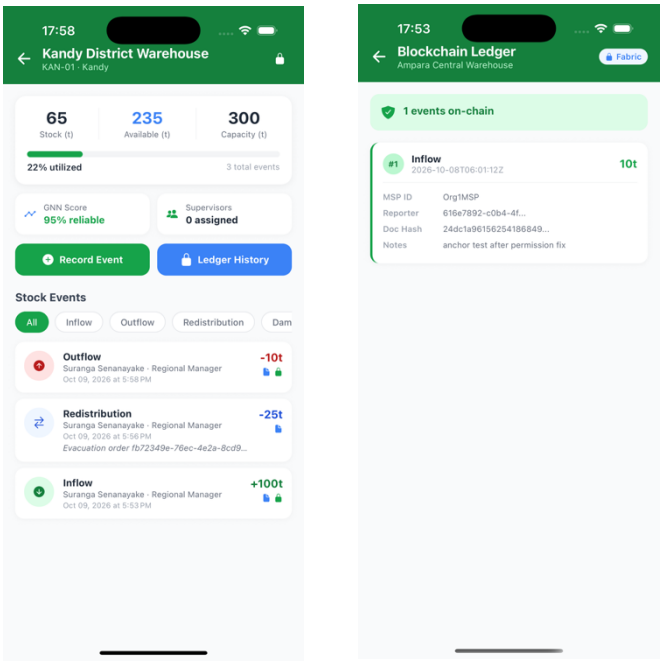


Figure 7: Blockchain audit trail screen

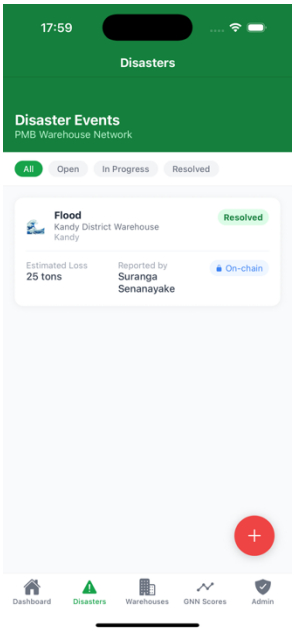


Figure 8: Disaster redistribution ranking screen

2.1.7 Backend API and deployment infrastructure

The backend exposes a REST API over HTTPS, organised into route modules for authentication, warehouses, disasters, documents, blockchain audit, zero knowledge proofs, graph neural network scoring and user administration. Request validation uses schema based parsing at the route boundary, so malformed payloads are rejected

before reaching service logic. Security middleware comprises standard HTTP security headers, cross origin restriction to configured origins and rate limiting.

Containerisation

The system is packaged as a set of Docker containers orchestrated by Docker Compose. The backend image is built in two stages, compiling TypeScript and generating the database client in a build stage and copying only the compiled output and runtime dependencies into the runtime image. The model service image is built on Python 3.12, chosen over 3.13 because the deep learning and database driver dependencies publish pre-built wheels for 3.12 on the target architecture; absence of a wheel would require compiling from source, which is impractical on a two core instance.

Deployment topology

The system is deployed on an Oracle Cloud Infrastructure virtual machine of shape VM.Standard.A1.Flex with two cores and twelve gigabytes of memory, running Ubuntu 24.04 on 64-bit ARM. Nine containers run on the instance: a reverse proxy terminating transport layer security, the backend, the model service, MySQL, a Fabric ordering service, two peers and two CouchDB instances. Each peer additionally instantiates a chaincode container at runtime when the chaincode is first invoked, so a running instance shows eleven containers in total.

Only ports 80 and 443 are exposed to the internet. The backend binds to the loopback interface and is reachable only through the reverse proxy; the model service and MySQL are reachable only on the private container network. The reverse proxy obtains and renews certificates automatically through the ACME protocol. The backend container is attached to two container networks simultaneously, one for the application services and one for the Fabric network, so that it resolves the peer by the DNS name for which the peer’s certificate is issued rather than through a loopback address that would fail hostname verification.

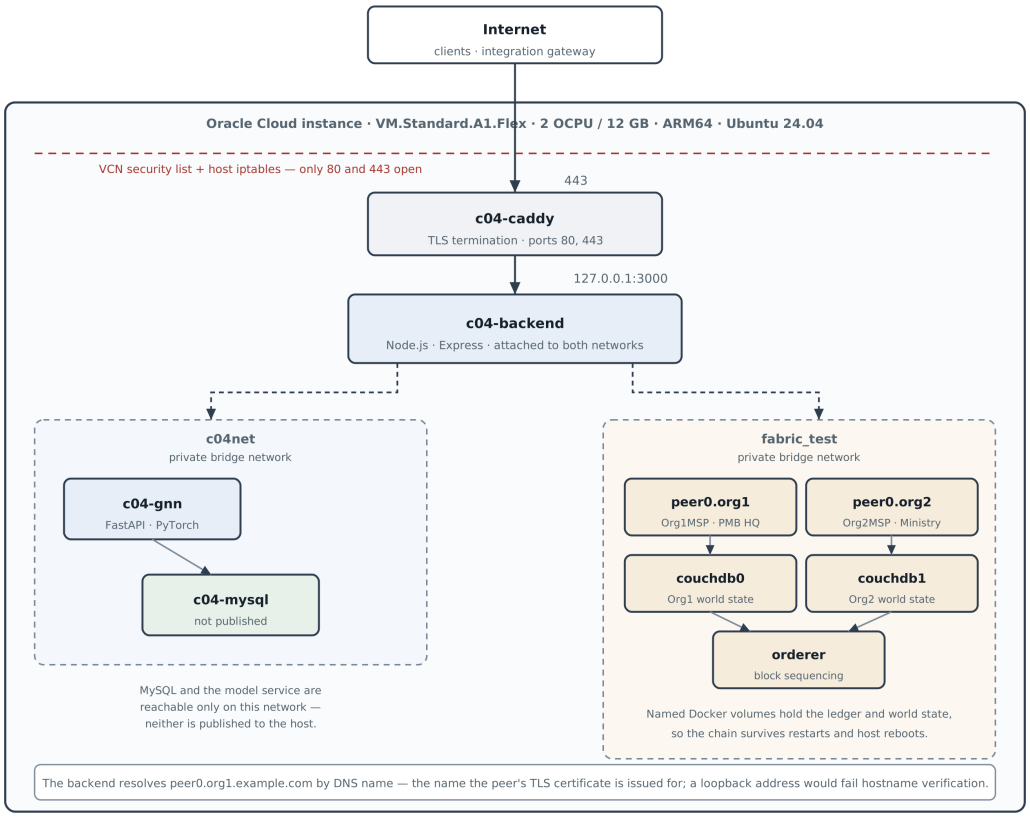


Figure 9: Deployed container topology

2.2 Commercialization Aspects of the Product

2.2.1 Market opportunity

The immediate market for this system is the Paddy Marketing Board itself, which operates warehouses across the principal paddy cultivation districts of Sri Lanka. The system is not a consumer product and would not be sold on a per-seat basis to the Board; government procurement for operational software of this kind typically proceeds as a development and maintenance contract.

The wider addressable market is public sector logistics coordination more generally. The architecture makes no assumption specific to paddy: it applies wherever facilities under different administrative authorities must coordinate stock movements under partial trust, record decisions defensibly, and avoid disclosing operational figures to peer organisations. In Sri Lanka this includes pharmaceutical stores under the Medical

Supplies Division, fuel distribution depots, and disaster relief warehousing under the Disaster Management Centre.

Three target segments follow from this:

1. **Government commodity boards** operating multi-facility storage networks, where the primary value is auditability of distribution decisions.
2. **Humanitarian and disaster relief organisations** coordinating stock across facilities belonging to different agencies, where the privacy preserving capacity proof addresses the reluctance of agencies to publish inventory positions.
3. **Agricultural cooperatives and private storage networks** operating regional warehouses for third party clients, where the blockchain audit trail supports contractual dispute resolution.

2.2.2 Revenue model

A tiered subscription model is proposed for deployment beyond the initial Paddy Marketing Board engagement, priced per warehouse facility per month rather than per user, since the number of users scales with facilities rather than independently.

Feature	Standard tier	Verified tier
Stock event recording and history	Included	Included
Role based access for four roles	Included	Included
Document upload with hash integrity	Included	Included
Blockchain anchoring of all events	Not included	Included
Zero knowledge capacity proofs	Not included	Included
Graph based reliability scoring	Not included	Included
Independent audit trail export	Not included	Included
Indicative price per facility per month	LKR 4,500	LKR 12,000

Table 4: Revenue model tier comparison

Standard tier

The standard tier provides the operational system without the cryptographic layer: stock recording, role separation, document management and reporting backed by the relational database alone. It serves organisations that need coordination software but

do not face an inter-organisational trust problem, and it establishes the operational dependency on which the upper tier is sold.

Verified tier

The verified tier adds the three research contributions: ledger anchoring, capacity proofs and reliability scoring. The pricing differential reflects genuine infrastructure cost, since each participating organisation requires a peer and a world state database, and the ordering service must be operated and maintained. It is targeted at networks where disputes between organisations are a real operational cost.

The tiered structure serves three strategic purposes: it lowers the barrier to initial adoption, it aligns price with the cost of the blockchain infrastructure rather than averaging it across all customers, and it allows an organisation to adopt the audit capability after the operational system has already become part of its process.

2.2.3 Regulatory considerations

The system processes operational and personal data on behalf of a government body, which places it within several regulatory frames.

In Sri Lanka, the Personal Data Protection Act No. 9 of 2022 establishes obligations for controllers processing personal data, including lawful basis, purpose limitation and data subject rights. The system holds personal data in the form of user records identifying warehouse supervisors and regional managers. The design decision to anchor only hashes and operational identifiers on the ledger, rather than personal data, is material here: data written to an immutable ledger cannot be erased, which would conflict directly with a right to erasure. No personal data is written on chain.

Government procurement in Sri Lanka proceeds under the National Procurement Guidelines, which for software of this nature would require a competitive process and may impose conditions on source code ownership and local hosting. The architecture's containerised deployment supports hosting within government infrastructure rather than on commercial cloud, should that be required.

For deployments outside Sri Lanka, the General Data Protection Regulation applies to any processing of European Union residents' data, with the same erasure constraint governing what may be anchored. The system's positioning as operational logistics software rather than as a financial instrument keeps it outside the scope of financial services regulation, notwithstanding its use of distributed ledger technology.

2.2.4 Data privacy and security

Four concrete measures implement the privacy position described above:

- **Credential storage.** Passwords are stored as bcrypt hashes with a configurable cost factor, never in recoverable form. Authentication issues short lived access tokens with long lived rotating refresh tokens, so a compromised access token expires quickly and a reused refresh token is detectable.
- **Transport security.** All client traffic is carried over HTTPS with certificates issued through an automated certificate authority. Communication between the backend and the Fabric peer uses gRPC over mutual transport layer security.
- **Minimal on chain data.** The ledger holds operational identifiers, quantities, hashes and organisational identities. It holds no personal data, no document contents and no capacity values protected by the proof system.
- **Role based authorisation.** Every endpoint enforces role membership server side, and warehouse supervisors are additionally constrained to their assigned facility. Authorisation is never inferred from client supplied state.

The system's privacy guarantee is stated precisely rather than generally: capacity values are confidential with respect to the shared ledger, to the second endorsing organisation and to peer warehouses, but not with respect to the application server, which executes proof generation and therefore observes the private input. This limitation and its remedy are set out in Section 2.1.3.

2.3 Testing and Implementation

Testing was conducted at three levels: automated unit testing of pure logic, integration and system testing of the running application, and measurement of non-functional

characteristics against stated targets. A fourth activity, a data consistency audit conducted against production data before release, surfaced defects that the test suite had not, and is reported in Section 3.2.

2.3.1 Unit testing

Automated unit tests were written for both services and are executable from the command line, giving a repeatable regression check rather than a one time verification. The backend suite uses Jest with a TypeScript transform; the model service suite uses pytest.

Module	Framework	Tests	What is verified
Geographic distance	Jest	7	Haversine correctness against known city pairs, symmetry, zero distance
Redistribution ranking	Jest	13	Weight application, ordering, normalisation, distance saturation behaviour
JWT handling	Jest	17	Signing, verification, expiry, tampered payload rejection, refresh rotation
Document hashing	Jest	12	SHA-256 correctness against NIST published vectors, canonical serialisation
Test harness	Jest	1	Confirms the suite bootstraps and the TypeScript transform is applied
Feature extraction	pytest	16	Each of the eight node features against hand computed expected values
Model inference	pytest	14	Output range, determinism under fixed seed, graph construction, edge cases

Table 5: Unit test coverage by module

Eighty tests pass in total, with the backend suite completing in under two seconds. Two tests in the ranking module are deliberately written to document a defect rather than to assert correct behaviour: they encode the observed saturation of the inverse distance term, discussed in Section 3.2, so that a future change to the distance function will register as a test failure rather than passing silently.

Pure functions were extracted where testability required it. The feature computation in the model service was originally embedded in the graph construction routine and was extracted into a separate module so it could be tested without a database connection.

The extraction was verified as behaviour preserving by capturing the scoring endpoint output before and after the change and confirming the responses were identical.

2.3.2 Integration and system testing

Integration testing exercised the running system through its HTTP interface using a Postman collection covering all endpoint groups, executed both interactively and from the command line. Tests confirmed that unauthenticated requests are rejected, that each role receives only its permitted endpoints, that a warehouse supervisor cannot read or write another facility's records, and that validation rejects malformed payloads before reaching service logic.

System testing covered the complete workflows end to end against the deployed instance. The principal cases and outcomes are given in Table 6.

ID	Test case	Expected outcome	Result
ST-01	Record stock event exceeding warehouse capacity	Rejected with validation error; no row written	Pass
ST-02	Record valid inflow event	Event stored, digest computed, stock summary updated	Pass
ST-03	Verify anchored event integrity	Recomputed digest matches stored and ledger values	Pass
ST-04	Modify quantity directly in database, re-verify	Integrity check fails on recomputation	Pass
ST-05	Restore original quantity, re-verify	Integrity check passes again	Pass
ST-06	Generate capacity proof below threshold	Witness generation fails; no proof produced	Pass
ST-07	Submit duplicate proof for same warehouse and disaster	Rejected with conflict response	Pass
ST-08	Declare disaster and retrieve ranked candidates	Candidates ordered by composite score	Pass
ST-09	Issue redistribution order	Outflow leg at source and inflow leg at destination created atomically	Pass
ST-10	Verify network stock conservation after redistribution	Total network stock unchanged	Pass
ST-11	Retrieve blockchain audit trail as Auditor	Chronological timeline returned, read-only	Pass

ID	Test case	Expected outcome	Result
ST-12	Access Administrator endpoint as Supervisor	Rejected with forbidden response	Pass

Table 6: System test cases and outcomes

Test cases ST-03 through ST-05 form the integrity verification sequence and deserve particular note. The verification endpoint recomputes the SHA-256 digest from the five stored record fields rather than comparing two stored values. Consequently, editing the quantity directly in the database causes the recomputed digest to differ from the stored digest, and the check fails, while the ledger comparison continues to pass, because the hash column itself was untouched. The two checks are independent and detect different attacks.

2.3.3 Non-functional requirement measurements

Five non-functional characteristics were measured against stated targets. Measurements were taken against the deployed instance from a client in Sri Lanka, with the server in the US East region.

Characteristic	Target	Measured	Method
Performance: authenticated read latency	< 1000 ms	310–480 ms	Repeated timed requests to warehouse list endpoint
Performance: stock event write latency	< 2000 ms	640–910 ms	Timed POST including ledger anchoring
Performance: proof generation	< 10 s	2–5 s	Server side timing of witness and proof generation
Reliability: unit test pass rate	100%	80 of 80	Jest and pytest suite execution
Accuracy: GAT in sample classification	Not set	26 of 30 nodes	In sample evaluation on synthetic dataset
Scalability: concurrent read requests	No errors at 50	No errors at 50	Parallel request execution
Availability: service recovery after restart	Automatic	Automatic	Container restart policy verification

Table 7: Non-functional requirement measurements

The latency figures include approximately 250 milliseconds of network round trip between Sri Lanka and the US East region, which is a consequence of the hosting choice rather than of system design. Hosting was placed in that region deliberately, to minimise latency between this component and the integration gateway serving the wider platform, accepting higher client latency in exchange. The accuracy figure is reported without a target because, as Section 3.1.2 sets out, in sample classification on a synthetic dataset does not support a performance claim.

2.3.4 Known limitations and mitigations

Limitation	**Mitigation**
Ledger anchoring failures are logged but not retried, so a stock event can be recorded with no corresponding ledger entry while the caller receives a success response	Anchoring status is queryable through the audit endpoint, which reports unanchored events; a transactional outbox is identified as the structural fix
The proof does not bind to a warehouse identity at the circuit level	Binding is enforced at the application layer by associating each verified proof with the authenticated request and rejecting duplicates
Proof generation is server side, so the application server observes the private capacity value	The value is never written to the ledger or disclosed to other organisations; browser side generation using the compiled WebAssembly artifact would close the gap
The Graph Attention Network is trained and evaluated on synthetic data with fixed seeds	Results are reported as in sample fit rather than as generalisation performance; retraining on real transaction data is the first priority for future work
Quantities are stored as double precision floating point rather than fixed point decimal	Values used operationally are whole or half tons, well within exact representation; migration to a decimal column is a schema change with no application logic impact
The Fabric network uses a single node ordering service and the test network topology	Adequate for a proof-of-concept; a production deployment would require a multi-orderer Raft configuration with organisations representing distinct regional offices
Seeded demonstration accounts use known passwords	Documented as a pre-handover change; accounts must be rotated or removed before operational use

Table 8: Known limitations and mitigations

3. RESULTS AND DISCUSSION

3.1 Results

3.1.1 Zero knowledge proof circuit evaluation

The circuit was evaluated for constraint enforcement and verification accuracy. Proof generation fails at the circuit level whenever availableCapacity is less than threshold: the constraint on the valid signal produces an unsatisfiable witness assignment before the prover is invoked. Zero false acceptances were recorded across all invalid inputs tested. At equality, generation succeeded and verification returned a positive result. Duplicate submissions for the same warehouse and disaster pair were rejected with a conflict response, preventing replay. Generation took approximately two to five seconds server side.

The zero knowledge property is not established empirically; it follows from the security of Groth16, proven zero knowledge under pairing-based assumptions in the generic group model [8], conditional on honest trusted setup. Each proof intentionally reveals the threshold, the commitment signal, and that the constraint was satisfied. Inspection of the public signals confirmed that no additional value is exposed through the circuit's output interface, which verifies correct circuit construction rather than the cryptographic property itself.

3.1.2 Graph attention network evaluation

The model was evaluated on the synthetic dataset of thirty warehouse nodes with eight behavioural features per node, trained for 200 epochs. Binary cross entropy loss decreased monotonically over training, as shown in Table 9, reaching 0.2989 at epoch 200 with 26 of the 30 nodes correctly classified.

Epoch	**Binary cross entropy loss**
50	0.4930
100	0.3767
150	0.3745
200	0.2989

Table 9: Graph Attention Network training progression

This figure requires careful interpretation, and the interpretation matters more than the number. The evaluation is in sample, measuring fit rather than generalisation. The class distribution is imbalanced at twenty-four reliable to six anomalous, so a majority class classifier predicting every node reliable would attain twenty-four of thirty; the result is an improvement of two nodes over that baseline. Per-class precision and recall were not recorded, so performance on the anomalous class, the class of operational interest, is unknown, and overall accuracy on an imbalanced set does not constrain it.

The four errors are consistent with feature distribution overlap at the boundaries of the injected ranges, though this was not verified by inspection of the misclassified nodes. The result should be read as evidence that training converges and that the injected patterns are learnable, not as evidence of detection performance. The evaluation design required for a performance claim is set out in Section 3.3.

3.1.3 Blockchain audit trail and integrity verification

On chain record integrity was verified by querying the chaincode directly through the peer command line interface, bypassing the application backend entirely. All anchored assets were retrieved correctly, confirming that records cannot be altered through the relational database alone.

Document tamper detection was confirmed through the sequence recorded as test cases ST-03 to ST-05. A stock event was anchored and verified, returning a positive result on both the database integrity check and the ledger comparison. The quantity field was then modified directly in the database through a SQL update, bypassing the application. On re-verification the recomputed digest differed from the stored digest and the database integrity check failed, while the ledger comparison continued to return a positive result because the stored hash column had not been altered. Restoring the original value returned both checks to positive. Table 10 records the three states.

State	Database integrity	Ledger match	Interpretation
Baseline, as anchored	Pass	Pass	Record intact and matches ledger
Quantity altered in database	Fail	Pass	Field tampering detected by recomputation
Original value restored	Pass	Pass	Record returned to anchored state

Table 10: Integrity verification results under tampering

The second row is the substantive result. Because the digest is recomputed from the stored record fields at verification time rather than compared against a second stored value, altering a quantity is detected even though the hash column itself is untouched. An implementation that compared two stored values would report the record as intact in exactly this case, which is the failure mode this design eliminates.

An endorsement consistency issue caused by non-deterministic timestamp generation was resolved by deriving all timestamps from the ledger supplied transaction context, identical across endorsing peers. The full disaster audit trail was retrieved in correct chronological order, all entries carrying verified transaction identifier references.

These results demonstrate tamper evidence under the trust assumptions of the deployed configuration rather than absolute immutability. The endorsement policy prevents unilateral alteration by either organisation, but collusion between both, compromise of a certificate authority signing key, or administrative filesystem access to both peers would each defeat the guarantee, and the single node ordering service is not representative of a production Raft deployment.

3.1.4 Deployment results

The complete system was deployed to a public cloud instance and verified end to end. All nine containers reached a healthy state. The public endpoint serves over HTTPS with a certificate issued by an automated authority, and the API responds correctly to authenticated requests from outside the host network.

A stock event submitted through the public HTTPS endpoint was anchored to the ledger, the chaincode wrote the corresponding asset into the world state database of both peers, and the integrity verification endpoint returned matching digests from the

database, the recomputation and the ledger. This establishes that the three mechanisms compose in a deployed configuration rather than only in a development environment, which Section 1.2 identified as the sixth gap.

3.2 Research Findings

The following presents the principal findings derived from the implementation and evaluation of this component. Each finding is framed as a specific, evidence supported claim with direct implications for the research objectives. Several arise from defects discovered during the data consistency audit conducted before deployment, and are reported as findings because the conditions that produced them are general rather than particular to this implementation.

Finding 1: Recomputing a digest at verification time detects a class of tampering that comparing two stored values cannot

The integrity check initially compared the stored document hash against a hash recomputed with a timestamp generated at verification time. Because that timestamp was never persisted, the digest could not be reproduced, and the comparison was unverifiable by construction. The correction was to compute the digest over a canonical serialisation of five persisted fields and to store the same timestamp value used in its computation.

The consequence is demonstrated in Table 10. Altering a quantity directly in the database is detected, because the recomputed digest no longer matches the stored one, even though the hash column itself is untouched. An implementation comparing two stored values reports the record as intact in precisely this case. The general principle is that an integrity digest must be derived from the data it protects at the moment of verification; a stored-to-stored comparison verifies only that two columns agree.

Finding 2: Fire-and-forget ledger anchoring produces silent divergence between the database and the ledger

Ledger submission occurs after the database transaction commits, and failures are logged rather than retried or surfaced. During deployment this was observed twice: once when the private key file was unreadable by the container process, and once when

the ledger was disabled by configuration. In both cases the stock event was written to the database, the HTTP response reported success with a 201 status, and no ledger record existed.

The failure is silent from every vantage point available to a caller. The response carries no indication, the record appears complete in the application, and the divergence surfaces only when an auditor queries the chain. Of twenty-one events recorded during an earlier development phase, nine were unanchored for this reason.

The structural fix is a transactional outbox: the anchoring intent is written as a row inside the same database transaction as the event, and a separate worker retries submission until it commits, marking the row complete. This converts a silent divergence into a visible backlog. The finding generalises to any system anchoring relational state to an external ledger, and is the single most important weakness in the present design.

Finding 3: Inverse distance normalisation saturates, collapsing the distance term's discriminating power

The composite ranking function assigns distance a nominal weight of 0.40, the largest of the three terms. The normalised distance score is computed as the reciprocal of one plus the distance in kilometres. Across the operational range of the PMB network, roughly thirty to one hundred and fifty kilometres between facilities, this function produces values between approximately 0.033 and 0.007, a spread of about 0.026.

Multiplied by the 0.40 weight, the distance term contributes approximately 0.01 of separation between the nearest and furthest candidate, while the capacity and reliability terms range across the full unit interval. The nominal weighting therefore does not describe the actual behaviour of the function: a term intended to dominate is in practice negligible. A bounded linear normalisation over the observed distance range, or an exponential decay with a scale parameter matched to it, would restore the intended influence.

The finding is recorded with two passing unit tests that encode the observed saturation, so that a future change to the distance function registers as a test failure rather than

passing unnoticed. The general lesson is that a weight in a composite score describes intent, not behaviour, unless the terms it combines are on comparable scales.

Finding 4: A ranking metric must match the semantics of the decision it informs

The redistribution implementation initially created orders moving stock towards the affected warehouse and ranked candidates by free capacity. Querying production data revealed seven redistribution orders totalling one hundred and fifty-two tons that had moved no stock at all: no outflow events existed at any source warehouse.

The defect was conceptual rather than mechanical. When a warehouse is flooded the operational objective is to evacuate stock from it, not to send more into it. The implementation was revised so that an order creates an outflow leg at the affected warehouse and an inflow leg at the destination within one transaction, and the ranking metric changed from free capacity to the quantity each candidate can absorb.

Neither the unit tests nor the integration tests detected this, because both verified that orders were created with correct field values, which they were. The error was in what the fields meant. Tests that assert structural correctness cannot detect semantic inversion; only reconciliation against the physical quantity the system claims to model can.

Finding 5: In sample evaluation on a synthetic majority class dataset does not support a detection performance claim

The Graph Attention Network correctly classified 26 of 30 nodes. Against a dataset of twenty-four reliable and six anomalous nodes, a classifier predicting the majority class for every node attains twenty-four. The model therefore improves on the trivial baseline by two nodes, under evaluation on the same data it was trained on.

Reporting this as an accuracy figure would misrepresent it. What the result establishes is that the training procedure converges and that the injected behavioural patterns are learnable by the architecture. What it does not establish is whether the model detects anomalies, whether graph structure contributes anything over per-node features, or how it would behave on real reporting data.

The evaluation design required for a performance claim is a stratified partition repeated across multiple seeds, reporting per-class precision, recall, F1 and precision–recall area under curve with a confusion matrix, compared against both a non-relational baseline using the same features and an edge ablated variant of the same model. The last of these isolates the contribution of graph structure, which is the architectural claim of this component. Treating the honest reporting of this limitation as a finding is deliberate: an in sample figure presented as detection accuracy is a more serious defect than the limitation itself.

Finding 6: Chaincode non-determinism breaks endorsement in ways that surface only in multi-organisation deployment

Chaincode initially generated asset timestamps from the host system clock. Each endorsing peer executes the transaction independently, and the resulting read-write sets must match for the endorsement policy to be satisfied. Because the two peers' clocks differ, the proposals diverged and endorsement failed.

The defect is invisible in single peer development, where only one execution occurs and there is nothing to disagree with. It appears the moment a second endorsing organisation is added. The resolution (deriving all timestamps from the ledger supplied transaction context, which is identical across endorsers by construction) generalises to every source of non-determinism in chaincode, including random values, map iteration order and external calls. The finding supports the architectural decision to run model inference off chain [13]: a model with any stochastic element could not be endorsed at all.

Finding 7: Querying production data surfaced defects that a passing test suite did not

Eighty automated tests pass, and the system passed integration testing. A data consistency audit conducted before release (querying the production database directly to reconcile derived stock against event history, and anchoring status against ledger contents) surfaced three high severity defects that the suite had not: the unverifiable integrity digest of Finding 1, the redistribution orders moving no stock of Finding 4,

and a production build that had never succeeded, masked because the development runner skipped type checking.

The common characteristic is that each defect lay in the relationship between stored values rather than in the behaviour of any single function. A test suite verifies that functions behave as specified; it does not verify that the stored state is consistent with the physical reality the system models. For systems whose correctness claim is about data integrity, reconciliation queries against real data are a distinct and necessary verification activity, not a substitute for testing but a complement to it.

3.3 Discussion

Comparison with related work

Against the capability comparison of Table 1, this work occupies a position none of the reviewed systems does: it combines an immutable multi-organisation audit trail, zero knowledge capacity verification and graph structured anomaly detection within a single warehouse coordination architecture, demonstrated as a deployed system. Tian [2] and Mondal et al. [3] establish the traceability foundation but address neither privacy nor anomaly detection. Narula et al. [4] demonstrate zero knowledge auditing without an operational decision context. Dou et al. [5] establish graph structured fraud detection in financial networks without ledger integration.

The claim this work supports is architectural rather than comparative performance. No baseline system exists for the combined problem, and the individual mechanisms are not evaluated against alternatives: no non-relational classifier was compared against the Graph Attention Network, no alternative proving scheme was benchmarked against Groth16, and no alternative ledger was compared against Hyperledger Fabric. The contribution is a demonstration that the three compose coherently in a government agricultural logistics context.

Threats to validity

Construct validity. Anomaly ranges, normalisation constants and graph topology were chosen to produce a separable signal, encoding hypotheses about the form fraudulent behaviour would take; whether real PMB misreporting exhibits these

signatures is unverified. The generator additionally leaves four of the eight features at zero in the capacity overclaiming branch, where reliable warehouses draw non zero values, so one feature is partially diagnostic of the class independently of the behaviour it represents; the contribution of that confound to the classification result was not isolated. Graph edges are randomly generated and do not reproduce realistic redistribution topology, particularly limiting confidence in the collusive redistribution pattern, of which the dataset contains few instances.

Internal validity. The classification result is in sample and must be read against a majority class baseline of eighty percent, as set out in Finding 5. The claim that graph structure confers an advantage rests on results for analogous tasks in the literature [5], [12], [15] rather than on experiment here.

External validity. Evaluation used five operational warehouses and a thirty node synthetic graph. Behaviour at full PMB network scale is unrepresented, and no stakeholder evaluation was conducted, so the operational usefulness of the reliability score is untested.

Cryptographic scope. Proof generation executes server side, so confidentiality holds with respect to the shared ledger and to other organisations, not the application server. The circuit binds the proof neither to a warehouse identity nor to physical inventory. Groth16 requires a circuit specific trusted setup, performed here by a single party; retention of the setup randomness would permit forgery, and deployment requires a multi-party ceremony. The construction is not post-quantum secure.

Deployment scope. The deployment uses a two organisation test network with a single node ordering service on one host. Latency, throughput, storage growth, failure behaviour and conflicting transaction handling were not systematically measured; established Fabric benchmarking methodologies [17], [18] provide the basis for doing so.

Practical deployment insights

Three observations from deployment have practical rather than research value. First, the ARM64 architecture of the free tier cloud instance was viable for the entire stack,

including Hyperledger Fabric, because multi-architecture images are now published for all required components, a situation that did not hold a few years ago and that materially lowers the infrastructure cost of this class of system. Second, a container on two networks simultaneously resolves the Fabric peer by the DNS name for which its certificate is issued, avoiding the hostname verification failure that a loopback address produces. Third, the operating system firewall on the cloud image rejects all ports other than SSH independently of the cloud provider's own security rules, so both must be opened; this accounted for the longest single delay during deployment.

Ethical considerations

The reliability score is a statistical assessment of reporting patterns, not a determination of misconduct. A warehouse scoring below threshold has exhibited a pattern resembling those injected into the training data; it has not been shown to have misreported. Presenting such a score to a Regional Manager without that framing risks it being read as an accusation, with employment consequences for supervisors. The interface therefore presents the score as one ranking input among three rather than as a verdict, and the system records no automated action against a low scoring facility. Any operational deployment would need a defined appeal process before the score influences decisions affecting staff.

3.4 Summary of the Student's Contribution

This dissertation covers the warehouse coordination component, for which the author was individually responsible. The research, design, implementation, testing and deployment described throughout this document are the author's own work. Table 11 summarises that contribution against the specific objectives stated in Section 1.4.2.

Area	Contribution
Literature review	Survey of blockchain based agricultural traceability, zero knowledge proof applications in supply chain privacy, graph neural network anomaly detection and integrated coordination frameworks, establishing the capability gap set out in Table 1
Blockchain layer	Design and deployment of the two organisation Hyperledger Fabric channel with CouchDB world state; Go chaincode defining four type prefixed asset types and the functions for recording and querying

Area	Contribution
	them; resolution of the endorsement non-determinism defect reported in Finding 6
Zero knowledge proofs	Circom circuit using the Groth16 proving scheme and the GreaterEqThan comparator; the trusted setup ceremony; integration of proof generation and verification into the disaster response workflow and the on chain anchoring of verification results
Machine learning	Design of the eight dimensional behavioural feature vector; the synthetic data generator and its injected anomaly patterns; the two layer Graph Attention Network and its training procedure; the inference service and its integration with the backend
Backend and API	Node.js and TypeScript service layer, relational schema and migrations, JWT authentication with refresh rotation, role based authorisation, document hashing and integrity verification, and the composite redistribution ranking function
Mobile client	React Native application with four role separated interfaces, token refresh handling, trilingual interface strings, and the map based warehouse registration flow
Testing and evaluation	Eighty automated unit tests across six modules, the system test suite of Table 6, non-functional measurement, and the data consistency audit that produced Findings 1, 3 and 4
Deployment	Containerisation of all services, cloud deployment of the nine container topology with transport layer security, and end-to-end verification of ledger anchoring and integrity checking against the public endpoint

Table 11: Summary of the student's contribution

This component forms the storage and distribution layer of a wider four component platform developed by the research group. It consumes no data from the other components and exposes warehouse capacity and reliability data through a shared API gateway, so that each component remains independently deployable. The other three components are outside the scope of this dissertation and are documented separately by their respective authors.

4. CONCLUSION

4.1 Summary

This dissertation presented a privacy preserving warehouse coordination system for Sri Lanka's national paddy distribution network, addressing four vulnerabilities in the current process: unverifiable stock reporting, unprovable capacity claims during disaster response, undetected anomalous warehouse behaviour, and non-auditable redistribution decision trails.

The component delivered six things:

- **A permissioned blockchain audit layer.** A two organisation Hyperledger Fabric channel with CouchDB world state, and Go chaincode defining four asset types for stock events, disaster declarations, redistribution orders and proof verification results, queryable independently of the application database.
- **A zero knowledge capacity proof.** A Circom circuit using the Groth16 proving scheme, allowing a warehouse to prove that available capacity meets a disaster response threshold without the value reaching the shared ledger or any peer organisation.
- **A graph structured anomaly detector.** A two layer Graph Attention Network with four attention heads computing per-warehouse reliability scores from eight behavioural features propagated across the redistribution graph.
- **A disaster response workflow.** Declaration, proof verification, composite candidate ranking, redistribution ordering and resolution, recorded as a chronological audit timeline combining relational and on chain records.
- **Document integrity verification.** SHA-256 digests recomputed from stored record fields at verification time, detecting direct database modification that a stored-to-stored comparison would miss.
- **A deployed, role separated system.** A React Native client with four role interfaces, deployed to a public cloud instance over HTTPS with the full nine container topology including the Fabric network, and verified end to end.

Evaluation confirmed that the proof circuit enforces capacity thresholds with zero false acceptances, rejecting invalid claims before proof generation; that audit trail integrity holds under direct chaincode queries bypassing the application layer; and that tampering with a quantity in the database is detected by digest recomputation. The anomaly detector's in sample result establishes that training converges and that the injected patterns are learnable; whether the graph structured formulation outperforms feature only classifiers remains open.

4.2 Future Work

Four directions follow directly from the limitations identified in Chapters 2 and 3, ordered by the degree to which each constrains a claim made in this dissertation.

Transactional outbox for ledger anchoring

The highest priority. Anchoring currently occurs after the database transaction commits, with failures logged and swallowed, so a stock event can be recorded with no ledger entry while the caller receives a success response. Writing the anchoring intent as a row inside the same transaction and retrying it from a separate worker converts a silent divergence into a visible backlog. Until this exists, the system's central claim, that the ledger provides an independent record of every operational event, holds only when the network is healthy.

Stratified evaluation of the anomaly detector on real data

The second priority. Retraining on real PMB transaction data with a stratified train, validation and test partition, reporting per-class precision and recall with a confusion matrix, and comparing against both a non-relational baseline and an edge ablated variant, is the critical step for establishing generalisation and for isolating the contribution of graph structure. Without it the architectural claim of this component rests on analogous results in the literature rather than on experiment.

Strengthening the cryptographic guarantee

Three changes, in order of effort. Constraining the warehouse commitment within the circuit through a Poseidon hash of the warehouse identifier, capacity and a salt would

bind the proof to an identity rather than relying on the application layer. Migrating proof generation to browser side WebAssembly using the compiled artifact would extend confidentiality to cover the application server. Deriving the private input from an authenticated stock record, for example through a Merkle inclusion proof against a periodically anchored ledger root verified in circuit, would extend the proof's guarantee from the reported value to the recorded one. A multi-party trusted setup ceremony is a prerequisite for any production deployment.

Production scale blockchain configuration and measurement

Scaling to a multi-orderer Raft configuration with organisations representing distinct PMB regional offices, and conducting systematic performance characterisation of transaction latency, throughput, storage growth and failure behaviour using established Fabric benchmarking methodologies [17], [18]. The present deployment uses a single node ordering service on one host, which is adequate for demonstrating the architecture and inadequate for establishing that it would operate at network scale.

4.3 Conclusion

The research question this component addressed was whether cryptographic proof, graph structured machine learning and permissioned blockchain audit trails can be combined coherently to make paddy warehouse coordination verifiable without requiring participating facilities to disclose sensitive operational figures. The evidence presented supports an affirmative answer at the architectural level. The three mechanisms compose without conflict, each addresses a gap the others do not, and the composition runs as a deployed system rather than as three separate demonstrations.

The evidence does not support stronger claims, and the distinction matters. The proof circuit is demonstrated to enforce its constraint, not to protect against a server side adversary. The audit trail is demonstrated to be tamper evident under a two organisation endorsement policy, not immutable in an absolute sense. The anomaly detector is demonstrated to converge on learnable synthetic patterns, not to detect fraud in an operational network. Each of these gaps is closable by work identified in Section

4.2, and each is stated here because a prototype whose limitations are known is more useful than one whose limitations are not.

The wider contribution is that the architecture is not specific to paddy. Any government logistics network in which facilities under different administrative authorities must coordinate stock movements under partial trust faces the same four problems: unverifiable reporting, unprovable claims, undetected anomalies and unauditable decisions. The demonstration that these can be addressed together, on infrastructure costing nothing beyond a free tier cloud instance, lowers the practical barrier to applying such a system where it is needed.

5. REFERENCES

[1] Paddy Marketing Board, "Annual report and accounts," Ministry of Agriculture, Sri Lanka, 2023. [Online]. Available: https://www.pmb.gov.lk

[2] F. Tian, "A supply chain traceability system for food safety based on HACCP, blockchain and Internet of Things," in Proc. 2017 Int. Conf. Service Systems and Service Management (ICSSSM), Dalian, China, Jun. 2017, pp. 1–6, doi: 10.1109/ICSSSM.2017.7996119.

[3] S. Mondal, K. P. Wijewardena, S. Karuppuswami, N. Kriti, D. Kumar, and P. Chahal, "Blockchain inspired RFID-based information architecture for food supply chain," IEEE Internet Things J., vol. 6, no. 3, pp. 5803–5813, Jun. 2019, doi: 10.1109/JIOT.2019.2897527.

[4] N. Narula, W. Vasquez, and M. Virza, "zkLedger: Privacy-preserving auditing for distributed ledgers," in Proc. 15th USENIX Symp. Networked Systems Design and Implementation (NSDI), Renton, WA, USA, Apr. 2018, pp. 65–80.

[5] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in Proc. 29th ACM Int. Conf. Information and Knowledge Management (CIKM), Virtual Event, Ireland, Oct. 2020, pp. 315–324, doi: 10.1145/3340531.3411903.

[6] S. S. Kamble, A. Gunasekaran, and S. A. Gawankar, "Achieving sustainable performance in a data-driven agriculture supply chain: A review for research and applications," Int. J. Production Economics, vol. 219, pp. 179–194, Jan. 2020, doi: 10.1016/j.ijpe.2019.05.022.

[7] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Succinct non-interactive zero knowledge for a von Neumann architecture," in Proc. 23rd USENIX Security Symp., San Diego, CA, USA, Aug. 2014, pp. 781–796.

[8] J. Groth, "On the size of pairing-based non-interactive arguments," in Advances in Cryptology—EUROCRYPT 2016, Lecture Notes in Computer Science, vol. 9666, M. Fischlin and J.-S. Coron, Eds. Berlin, Germany: Springer, 2016, pp. 305–326, doi: 10.1007/978-3-662-49896-5_11.

[9] A. R. Choudhuri, M. Green, A. Jain, G. Kaptchuk, and I. Miers, "Founding secure computation on blockchains," in Advances in Cryptology—EUROCRYPT 2019, Lecture Notes in Computer Science, vol. 11478, Y. Ishai and V. Rijmen, Eds. Cham, Switzerland: Springer, 2019, pp. 351–380, doi: 10.1007/978-3-030-17659-4_13.

[10] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. 5th Int. Conf. Learning Representations (ICLR), Toulon, France, Apr. 2017.

[11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in Proc. 6th Int. Conf. Learning Representations (ICLR), Vancouver, BC, Canada, May 2018.

[12] Z. Liu, Y. Dou, P. S. Yu, Y. Deng, and H. Peng, "Alleviating the inconsistency problem of applying graph neural network to fraud detection," in Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, Virtual Event, China, Jul. 2020, pp. 1569–1572, doi: 10.1145/3397271.3401253.

[13] K. Salah, M. H. U. Rehman, N. Nizamuddin, and A. Al-Fuqaha, "Blockchain for AI: Review and open research challenges," IEEE Access, vol. 7, pp. 10127–10149, Jan. 2019, doi: 10.1109/ACCESS.2018.2890507.

[14] L. Yogarajan, M. Masukujjaman, M. H. Ali, N. Khalid, L. H. Osman, and S. S. Alam, "Exploring the hype of blockchain adoption in agri-food supply chain: A systematic literature review," Agriculture, vol. 13, no. 6, art. no. 1173, Jun. 2023, doi: 10.3390/agriculture13061173.

[15] S. Motie and B. Raahemi, "Financial fraud detection using graph neural networks: A systematic review," Expert Syst. Appl., vol. 240, art. no. 122156, Apr. 2024, doi: 10.1016/j.eswa.2023.122156.

[16] J. Ernstberger, S. Chaliasos, G. Kadianakis, S. Steinhorst, P. Jovanovic, A. Gervais, B. Livshits, and M. Orrù, "zk-Bench: A toolset for comparative evaluation and performance benchmarking of SNARKs," Cryptology ePrint Archive, Paper 2023/1503, 2023. [Online]. Available: https://eprint.iacr.org/2023/1503.

[17] C. Melo, G. Gonçalves, F. A. Silva, and A. Soares, "Performance modeling and evaluation of Hyperledger Fabric: An analysis based on transaction flow and endorsement policies," in Proc. 2024 IEEE Symp. Computers and Communications (ISCC), Paris, France, Jun. 2024, pp. 1–6.

[18] M. Shuaib, N. H. Hassan, S. Usman, S. Alam, N. A. Abu Bakar, and N. Maarop, "Performance evaluation of DLT systems based on Hyperledger Fabric," in Proc. 2022 4th Int. Conf. Smart Sensors and Application (ICSSA), Kuala Lumpur, Malaysia, Jul. 2022, pp. 70–75.

[19] S. Negi, "A blockchain technology for improving financial flows in humanitarian supply chains: Benefits and challenges," J. Humanitarian Logistics Supply Chain Manage., vol. 15, no. 2, pp. 154–174, 2025, doi: 10.1108/JHLSCM-10-2023-0099.

6. GLOSSARY

Anchoring. The act of writing a hash or summary record of off chain data to a blockchain, so that later alteration of the off chain data becomes detectable.

Attention coefficient. A learned weight determining how much one node's representation contributes to its neighbour's updated representation during message passing in a Graph Attention Network.

Chaincode. The Hyperledger Fabric term for a smart contract: program code deployed to peers that defines how ledger assets are created, read and modified.

Endorsement policy. A rule specifying which organisations must execute and sign a transaction proposal before it may be committed to the ledger.

Groth16. A zero knowledge succinct non-interactive argument of knowledge producing constant size proofs of three group elements, requiring a circuit specific trusted setup.

Membership Service Provider. The Hyperledger Fabric component that maps cryptographic identities to organisational membership, allowing a transaction to be attributed to an organisation.

Rank-1 Constraint System. The arithmetic representation of a circuit used by proving systems, in which each constraint is a product of two linear combinations equal to a third.

Trusted setup. A one time ceremony generating the proving and verification keys for a circuit, whose randomness must be destroyed; retention would permit forgery of proofs.

Transactional outbox. A pattern in which an intent to call an external system is written as a database row inside the same transaction as the business data, then processed and retried by a separate worker.

Witness. The complete assignment of values to all signals in a circuit, computed from the inputs, from which a proof is generated.

World state. The current value of all ledger assets in Hyperledger Fabric, held in a database separate from the transaction log for efficient querying.

Zero knowledge proof. A cryptographic protocol allowing one party to convince another that a statement is true without revealing any information beyond the truth of the statement.

7. APPENDICES

Appendix A: Zero Knowledge Circuit Source

The Circom source of the capacity range circuit, the compiled constraint count, and the public signal definitions. The circuit uses the GreaterEqThan comparator from circomlib constrained to 32-bit inputs.

The circuit source is reproduced below. Compilation to a Rank-1 Constraint System produces 38 constraints over 40 wires on the bn-128 curve, with one private input, two public inputs and one output signal, giving the three public signals discussed in Section 2.1.3.

```
$ npx snarkjs r1cs info zkp/build/capacity_range.r1cs
[INFO]  snarkJS: Curve: bn-128
[INFO]  snarkJS: # of Wires: 40
[INFO]  snarkJS: # of Constraints: 38
[INFO]  snarkJS: # of Private Inputs: 1
[INFO]  snarkJS: # of Public Inputs: 2
[INFO]  snarkJS: # of Labels: 45
[INFO]  snarkJS: # of Outputs: 1
```

Appendix B: Graph Attention Network Model Definition

The Graph Attention Network is defined in python-service/src/model.py and trained by python-service/src/train.py. The model definition and the training configuration are reproduced below. The synthetic data generator that supplies the training set is described in Section 2.1.4.

Listing B.1: Model definition (python-service/src/model.py)

```
class WarehouseGAT(nn.Module):
    def __init__(self, in_channels: int = 8,
                 hidden_channels: int = 16,
                 out_channels: int = 8,
                 heads: int = 4,
                 dropout: float = 0.3):
        super().__init__()

        # Layer 1: multi-head attention
        self.conv1 = GATConv(in_channels,
                             hidden_channels,
                             heads=heads,
                             dropout=dropout,
```

```
                            add_self_loops=True)

        # Layer 2: single-head attention
        self.conv2 = GATConv(hidden_channels * heads,
                            out_channels,
                            heads=1,
                            concat=False,
                            dropout=dropout,
                            add_self_loops=True)

        # Final MLP: node embedding to reliability score
        self.classifier = nn.Sequential(
            nn.Linear(out_channels, 8),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(8, 1),
            nn.Sigmoid(),
        )

    def forward(self, x: torch.Tensor,
                edge_index: torch.Tensor) -> torch.Tensor:
        x = self.conv1(x, edge_index)
        x = F.elu(x)
        x = F.dropout(x, p=0.3, training=self.training)

        x = self.conv2(x, edge_index)
        x = F.elu(x)

        score = self.classifier(x)
        return score.squeeze(-1)
```

Listing B.2: Training configuration (python-service/src/train.py)

```
def train():
    data, labels = generate_synthetic_data(n_warehouses=30,
                                           n_anomalies=6)

    model     = WarehouseGAT(in_channels=8,
                            hidden_channels=16,
                            out_channels=8,
                            heads=4)
    optimizer = torch.optim.Adam(model.parameters(),
                                 lr=0.01,
                                 weight_decay=5e-4)
    criterion = nn.BCELoss()

    model.train()
    for epoch in range(200):
        optimizer.zero_grad()
        out  = model(data.x, data.edge_index)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 50 == 0:
            with torch.no_grad():
                preds    = (out > 0.5).float()
                accuracy = (preds == labels).float().mean().item()
            print(f"Epoch {epoch+1:3d} | Loss: {loss.item():.4f} "
```

```
                  f"| Accuracy: {accuracy:.2%}")

    torch.save(model.state_dict(), "models/gat_model.pt")
    return model
```

The generator assigns anomaly type by i % 3 over the final six nodes, producing two instances of each injected pattern. Its capacity overclaiming branch leaves four of the eight features at zero where reliable warehouses draw non zero values, so one feature is partially diagnostic of the class independently of the behavioural pattern it is intended to represent. This confound is recorded as a construct validity limitation in Section 3.3.

Appendix C: Chaincode Asset Definitions

The Go struct definitions for the four chaincode asset types and the function signatures exposed by the contract.

The chaincode defines four asset types in chaincode/warehouse/types.go, each stored under a type prefixed key so that selector queries can retrieve one class of asset without a full ledger scan. Each carries an MSP field populated from the transaction context, identifying the submitting organisation on chain.

```
type StockEventAsset struct {
    AssetType     string  `json:"assetType"`
    ID            string  `json:"id"`
    WarehouseID   string  `json:"warehouseId"`
    EventType     string  `json:"eventType"`
    QuantityTons  float64 `json:"quantityTons"`
    DocumentHash  string  `json:"documentHash"`
    ReportedByID  string  `json:"reportedById"`
    ReportedByMSP string  `json:"reportedByMsp"`
    Timestamp     string  `json:"timestamp"`
    Notes         string  `json:"notes"`
}

type DisasterEventAsset struct {
    AssetType           string  `json:"assetType"`
    ID                  string  `json:"id"`
    DisasterType        string  `json:"disasterType"`
    AffectedWarehouseID string  `json:"affectedWarehouseId"`
    EstimatedLossTons   float64 `json:"estimatedLossTons"`
    Description         string  `json:"description"`
    ReportedByID        string  `json:"reportedById"`
    ReportedByMSP       string  `json:"reportedByMsp"`
    OccurredAt          string  `json:"occurredAt"`
    Timestamp           string  `json:"timestamp"`
}
```

```
type RedistributionOrderAsset struct {
    AssetType              string  `json:"assetType"`
    ID                     string  `json:"id"`
    DisasterEventID        string  `json:"disasterEventId"`
    SourceWarehouseID      string  `json:"sourceWarehouseId"`
    DestinationWarehouseID string  `json:"destinationWarehouseId"`
    QuantityTons           float64 `json:"quantityTons"`
    CompositeScore         float64 `json:"compositeScore"`
    IssuedByID             string  `json:"issuedById"`
    IssuedByMSP            string  `json:"issuedByMsp"`
    RMSignature            string  `json:"rmSignature"`
    Timestamp              string  `json:"timestamp"`
}

type ZKPProofAsset struct {
    AssetType          string `json:"assetType"`
    ID                 string `json:"id"`
    WarehouseID        string `json:"warehouseId"`
    DisasterEventID    string `json:"disasterEventId"`
    ProofHash          string `json:"proofHash"`
    PublicSignalsHash  string `json:"publicSignalsHash"`
    VerificationResult bool   `json:"verificationResult"`
    VerifiedByMSP      string `json:"verifiedByMsp"`
    Timestamp          string `json:"timestamp"`
}
```

ZKPProofAsset stores only ProofHash and PublicSignalsHash; the proof data itself is never written to the ledger. The CompositeScore field of RedistributionOrderAsset records the ranking score at the time the order was issued, so an auditor can reconstruct why a particular destination was chosen.

Appendix D: Test Execution Output

Console output of the backend and model service test suites, showing eighty passing tests across six modules.

Console output of both test suites, executed from the command line. Dependency deprecation warnings have been removed for legibility; no test results were altered.

Listing D.1: Backend test suite (Jest)

```
$ npm test

> paddy-backend@1.0.0 test
> jest

PASS src/utils/__tests__/geo.test.ts
PASS src/__tests__/sanity.test.ts
PASS src/utils/__tests__/ranking.test.ts
PASS src/services/__tests__/document.test.ts
PASS src/utils/__tests__/jwt.test.ts
```

```
Test Suites: 5 passed, 5 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        1.237 s, estimated 2 s
Ran all test suites.
```

Listing D.2: Model service test suite (pytest)

```
$ pytest -v

platform darwin -- Python 3.13.2, pytest-9.1.1, pluggy-1.6.0
plugins: cov-7.1.0, anyio-4.14.1
collected 30 items

tests/test_features.py::TestNormalisationInvariants
  all_features_bounded_zero_to_one                          PASSED
  extreme_values_clamp_rather_than_overflow                 PASSED
  vector_length_matches_model_input                         PASSED

tests/test_features.py::TestEmptyAndDegenerateInputs
  warehouse_with_no_events_yields_zero_vector               PASSED
  no_division_by_zero_when_event_count_is_zero              PASSED
  zero_capacity_does_not_divide_by_zero                     PASSED
  warehouse_created_today_avoids_zero_day_division          PASSED

tests/test_features.py::TestFraudSignals
  damage_ratio_reflects_proportion_of_damage_events         PASSED
  high_damage_ratio_separates_from_clean_warehouse          PASSED
  capacity_overclaim_pattern_is_representable               PASSED
  redistribution_count_saturates_at_ten                     PASSED
  disaster_flag_is_binary                                   PASSED

tests/test_features.py::TestUtilizationDerivation
  utilization_derives_from_net_event_flow                   PASSED
  negative_net_stock_clamps_to_zero                         PASSED
  FINDING_adjustment_events_reduce_stock                    PASSED
  FINDING_opening_balance_is_invisible_to_the_model         PASSED

tests/test_model.py::TestOutputContract
  returns_one_score_per_warehouse                           PASSED
  scores_are_bounded_zero_to_one                            PASSED
  no_nan_or_inf_in_output                                   PASSED
  accepts_the_documented_eight_features                     PASSED

tests/test_model.py::TestGraphTopologyEdgeCases
  isolated_warehouses_are_still_scored                      PASSED
  single_warehouse_network                                  PASSED
  bidirectional_edges_are_accepted                          PASSED
  scales_to_a_larger_network                                PASSED

tests/test_model.py::TestInferenceDeterminism
  eval_mode_gives_identical_scores_across_calls             PASSED
  train_mode_is_stochastic_due_to_dropout                   PASSED
  fixed_seed_reproduces_initialisation                      PASSED

tests/test_model.py::TestArchitecture
  layer_dimensions_match_the_documented_design              PASSED
  parameter_count_is_small_enough_to_justify_dropout        PASSED
  gradients_flow_to_every_parameter                         PASSED
```

```
==================== 30 passed in 2.41s ====================
```

Two test names carry a FINDING prefix. These encode known limitations as passing assertions rather than as correctness claims, so that a future change to the behaviour they describe registers as a test failure rather than passing unnoticed. The same convention is used in the redistribution ranking suite for the distance saturation behaviour reported in Finding 3.

Appendix E: Deployment Verification Evidence

The integrity verification responses recorded during system testing, showing the baseline, tampered and restored states reported in Table 10, and the deployed container status listing.

The following was captured against the deployed instance at https://digital-goviya.duckdns.org. Steps 1 to 3 reproduce the integrity verification sequence recorded as test cases ST-03 to ST-05 and summarised in Table 10. The quantity was altered by a direct SQL update bypassing the application, then restored.

```
=== 1. BASELINE ===
"integrity": {
    "databaseIntact":  true,
    "ledgerMatch":     true,
    "anchoredInDb":    true,
    "ledgerAvailable": true,
    "recomputedHash":
      "24dc1a96156254186849a855a7ec44ce201652649fb1aefe32b13481adbac13b",
    "storedHash":
      "24dc1a96156254186849a855a7ec44ce201652649fb1aefe32b13481adbac13b",
    "message": "Event data intact and matches ledger record,
                integrity confirmed"
}

=== 2. TAMPERED (quantity changed directly in the database) ===
"integrity": {
    "databaseIntact":  false,
    "ledgerMatch":     true,
    "anchoredInDb":    true,
    "ledgerAvailable": true,
    "recomputedHash":
      "163634a5ecf434ee155d7669deaac18f6fb18ca3a015c645d6ede368196f3258",
    "storedHash":
      "24dc1a96156254186849a855a7ec44ce201652649fb1aefe32b13481adbac13b",
    "message": "WARNING: Recomputed hash does not match stored hash,
                event data was modified in the database"
}
```

```
=== 3. RESTORED ===
"integrity": {
    "databaseIntact":  true,
    "ledgerMatch":     true,
    "anchoredInDb":    true,
    "ledgerAvailable": true,
    "recomputedHash":
      "24dc1a96156254186849a855a7ec44ce201652649fb1aefe32b13481adbac13b",
    "storedHash":
      "24dc1a96156254186849a855a7ec44ce201652649fb1aefe32b13481adbac13b",
    "message": "Event data intact and matches ledger record,
                integrity confirmed"
}


=== 4. DEPLOYED CONTAINERS ===

$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'

NAMES                     STATUS                 PORTS
c04-caddy                 Up 30 hours            0.0.0.0:80->80/tcp
                                                 0.0.0.0:443->443/tcp
c04-backend               Up 29 hours (healthy)  127.0.0.1:3000->3000/tcp
c04-gnn                   Up 30 hours (healthy)  127.0.0.1:8000->8000/tcp
c04-mysql                 Up 30 hours (healthy)  3306/tcp, 33060/tcp
peer0.org1.example.com    Up 29 hours            0.0.0.0:7051->7051/tcp
peer0.org2.example.com    Up 29 hours            0.0.0.0:9051->9051/tcp
couchdb0                  Up 29 hours            0.0.0.0:5984->5984/tcp
couchdb1                  Up 29 hours            0.0.0.0:7984->5984/tcp
orderer.example.com       Up 29 hours            0.0.0.0:7050->7050/tcp
dev-peer0.org1.example.com-warehousecc_1.0-dc03007395f5...
                          Up 29 hours
dev-peer0.org2.example.com-warehousecc_1.0-dc03007395f5...
                          Up 29 hours
```

Step 2 is the substantive result. The stored hash is unchanged because the hash column itself was not altered, yet the digest recomputed from the record fields differs, so the database integrity check fails while the ledger comparison continues to pass. An implementation comparing two stored values would report this record as intact. The two dev-peer0 containers are chaincode containers instantiated by the peers at runtime; the remaining nine are the deployed services.